

UNO: Coordinator-Free Crash-Atomic Publication over Ordinary Blobs

Commonware storage design note

6 August 2026

Abstract

UNO is an append-oriented storage design for atomically publishing pending epochs of several ordinary blobs without a coordinator file, root WAL, or shared allocation layer. Payload is written once at its final offset. Each participant stores one checksummed local witness beside an otherwise invisible prepared root; witnesses form a canonical successor ring. The intended design treats a complete, valid ring as the commit decision and places participant durability operations in one serial barrier layer, even though each file retains its own durability boundary.

This paper specifies a target transition system using generation-colored root spellings under a crash model in which *any subset* of non-durable addressed writes may survive—not merely a prefix of submission order. It states the assumptions and invariants, gives a separate argument for rings broken after deletion finalization, derives bounded-recovery and acknowledged-durability targets, and identifies the cases outside the intended theorem. In particular, absence is never a prepare vote. No path may disappear until every participant has an independent, durable final root. Thereafter each retained root selects the new value and each tombstone independently denotes logical absence, so out-of-order cleanup can strand physical garbage but cannot expose a mixed vector. The checked-in implementation is compared against that target rather than assumed to satisfy it. The audit exposes concrete refinement blockers, including stale-ring authority and unconsumed rejected witnesses, and records the reviewed repair for a filesystem- backend cancellation defect; §13 states the distinction explicitly. Fresh adversarial review found a target-protocol blocker: a one-phase prepare can splice an older witness topology with newly issued candidate bytes into an exact, never-issued smaller ring. The unconditional crash-atomicity theorem is therefore withdrawn at this checkpoint. The result is a reviewable specification with an explicit open architecture choice, not a claim of proof or machine verification.

Status. UNO is an opt-in ALPHA API. Ordinary blob APIs and formats are unchanged. The append/rewind mechanism corresponds to the current code, while the root format below is a proposed R14 repair. The previously stated unconditional safety theorem is blocked even for that target: the one-phase prepare does not give exact witnesses issuance provenance under arbitrary byte-subset survival. Conjecture 1 is conditional on closing that gap. Current implementation deviations—including stale-ring authority, rejected-witness reuse, one integrity-boundary error, and unenforced direct-file/no-alias namespace preconditions—are listed in §13. That section separately records a reviewed cancellation-ownership repair; no R14 conformance claim follows from that narrower fix.

1 Problem and scope

Journal-shaped structures commonly need one logical mutation to change several append-only byte sequences: values and offsets, data and an index, or the active tails of several sections. If each file publishes independently, a crash can expose a mixed vector. A conventional coordinator WAL solves this by introducing another durable authority and at least one ordering edge. UNO instead places a local fragment of the decision in every participant and makes the fragments mutually discoverable.

The development name “UNO” denotes the target of writing payload *once* and using one hot-path barrier layer. It is not a claim that one system call or one physical device flush occurs: an n -participant group still has n per-file durability obligations. The obligations are independent and may be submitted concurrently.

1.1 Supported mutation vocabulary

The current public batch vocabulary is:

- **publish:** expose a participant’s pending append/tag epoch;
- **rewind:** publish a non-increasing encoded length, never an extension; and
- **remove:** delete the current exact blob incarnation.

Blob creation and whole-partition deletion are not group operations. Arbitrary positioned overwrite is also excluded: once a byte is committed, a later epoch may retain it or exclude it by rewinding, but may not replace it in place. This restriction is what permits write-once payload without undo data, page maps, hole management, or compaction.

1.2 Events, observations, and safety target

Let group G target the ordered logical-key set $K_G = \langle k_0, \dots, k_{n-1} \rangle$ and the exact physical participants $P_G = \langle p_0, \dots, p_{n-1} \rangle$ occupying those keys when G is validated. For each key, O_i is its caller-observable selected value—encoded length, integrity state, application tag, and exact encoded payload bytes—before G , while N_i is the corresponding candidate value ($\perp$ for a removal). Internal root generation, spelling, and witness bytes are deliberately outside this logical projection.

For any mutating operation H , $\text{admit}(H)$ is its first issued root, witness, or namespace update that can affect crash recovery after validation and lock acquisition. Staging an append or preflushing invisible payload is not admission because it cannot change a selected root. The following group events are distinct:

Admission $\text{admit}(G)$.

For a group, the generic admission event is the first byte of a G -specific prepared root or witness.

Decision $\text{decide}(G)$.

Either every participant prepare barrier has succeeded, or crash recovery has read and validated a complete exact ring including every required payload byte. Once established, this is a monotone protocol fact.

Acknowledgement $\text{ack}(G)$.

start_apply returns Ok. This transition is permitted only after the normal prepare-barrier route has established $\text{decide}(G)$.

Completion $\text{complete}(G)$.

The returned handle resolves successfully after the in-memory state is active and required eager materialization, truncation, and namespace work has finished. Completion is not the durability linearization point.

A failed attempt to establish $\text{decide}(G)$ is not the opposite monotone event. In particular, a recognizable candidate generation or non-quiescent higher-generation image that does not currently prove a candidate is only *provisionally rejected* until an exact code-last A root consumes that generation. A crash during the non-guard abort-body write may leave a different byte image that validates the complete candidate and therefore establishes the decision. Recovery returns the predecessor for caller mutation, truncates or overwrites payload, or reuses its slot only after exact A ; proof-preserving predecessor finalization remains permitted and may be required before A . The sole no- A case is a complete slot that is byte-for-byte the canonical lower-generation pre-attempt source, including any exact witness belonging to that lower generation, so no distinguishable candidate-generation admission effect remains to consume. Once exact A exists, the old outcome is irreversible for the consumed generation.

For a crash image D , $\text{view}_G(D)$ is the vector returned by a *successful* fresh namespace recovery followed by ordinary root recovery, before the caller optionally recre-

ates an absent key. A retained key maps to its selected metadata and payload bytes; an exact tombstone or a missing removed incarnation maps to $\perp$; internal generation/spelling is projected away. A recovery that detects corruption and fails closed returns no vector and cannot be mistaken for either outcome. Under the stated assumptions, a crash image created solely by protocol transitions has a defined recovery vector; fail-stop is reserved for an assumption violation or a subsequent I/O failure. For one fixed disk image recovery is deterministic; “old or new” below quantifies over the different disk images allowed by the crash model.

Definition 1 (Group crash atomicity). *After $\text{admit}(G)$ and before admission of any later create, publish, rewind, exact remove, or whole-partition remove whose target contains a key in K_G , every protocol-reachable crash image recovers to a vector satisfying*

$$\text{view}_G(D) \in \{ \langle O_0, \dots, O_{n-1} \rangle, \langle N_0, \dots, N_{n-1} \rangle \}.$$

An admitted but unacknowledged group may yield either vector across allowed crash images. Every image after $\text{ack}(G)$ must yield the new vector.

The definition is crash/reopen atomicity, not live read isolation. Existing handles may expose pending bytes according to the API, and recreation is a later key mutation rather than part of the recovered absence observation. After a later conflicting operation is admitted, the old/new-only interval ends; the stronger remaining obligation is that recovery correspond to a serial history containing G before that later operation. UNO intentionally defines no total durable order between disjoint groups. A semantic dependency must be placed in one group or ordered by waiting for the first acknowledgement.

2 System and fault model

2.1 Objects and identities

A storage namespace is exclusively owned by one storage instance and its clones. A logical key is $k = (q, x)$, where q is a UTF-8 partition string and x is an uninterpreted blob-name byte string. Keys are ordered lexicographically by q and then by raw bytes of x .

A physical participant is

$$p = (k, \iota, f),$$

where $\iota \in \{0, 1\}^{128}$ is the immutable incarnation in the file header and f is the current directly named regular-file inode at k ; following a symbolic link never yields a participant. Recreating the same key mints a new incarnation. Protocol records always identify both k and ι ; unlink additionally checks the opened and current device/inode identity. An ordinary non-atomic recreation may instead create a V1 blob with no UNO incarnation; it is a fresh named inode generation and cannot be mistaken for the exact V2 participant.

2.2 Medium operations

The proof uses the following abstract interface.

`write( $f, o, b$ )`

issues bytes b at file offset o .

`truncate( $f, L$ )`

issues a file-length change to L and preserves bytes below L .

`sync( $f$ )`

is a file durability barrier. On success, all earlier writes issued to f are durable together with the last earlier issued file length and the metadata required to retrieve them.

`unlink( $k$ )`

removes a directory entry but may not yet make that absence crash-durable.

`dirsinc( $q$ )`

makes earlier namespace changes in partition directory q durable.

A durable positioned write such as Linux `pwritev2(RWF_DSYNC)` is a scoped durability operation: on success at least its addressed bytes and required retrieval metadata enter the durable base. The implementation may persist unrelated dirty data too, but the proof takes no durability credit for it. UNO uses that weaker obligation only where the candidate does not depend on those unrelated bytes, such as a deletion candidate naming the prior committed payload. A write beyond the current end also issues the length extension needed to address its bytes. When a retained extension is reached in issue order, it first zero-initializes the offsets newly exposed at that point; retained addressed-byte updates belonging to the extending write are then applied. Thus if the extension survives without every byte of that write, the gap and the missing addressed cells read as zero, matching Linux hole and unwritten-extension semantics; this zero-fill is part of the length operation rather than invented medium damage. Bytes beyond the selected end remain inaccessible and cannot satisfy candidate validation. A selected shrinking truncate preserves the prefix and discards the old tail. Any later extension exposes zeros, and UNO assigns fresh write stamps rather than treating discarded tail contents as durability credit.

Assumption 1 (Arbitrary-subset addressed-write crash model). *For each file, a successful barrier establishes a durable base equal to applying every prior issued byte update and length change in issue order. At a later crash, the recovered file is obtained by starting from that base, retaining any subset of subsequently issued addressed-byte updates and length changes, and applying the retained updates in issue order. Selection is independent by byte, offset, and file; a multi-byte write may tear arbitrarily. Thus an overlapping byte is either its last barrier-established value or a value subsequently issued to that address, never a value superseded before the barrier. The medium does not invent values or damage an unaddressed byte. Directory-entry durability is independent: before a*

directory barrier, any subset of subsequent namespace updates may survive; after it, all earlier updates do. The recovered crash image becomes the durable base for the next execution, so a byte observed after restart is not again treated as a pre-crash volatile update.

This is strictly stronger than a prefix model: the second half of a root may survive while the first does not, and participant p_7 may survive while p_2 does not. Every protocol write, including recovery normalization, abort publication, final-root repair, truncation, and namespace cleanup, is subject to the same rule. Recovery has no atomic-write privilege.

Assumption 2 (Trusted local medium). *The disk and namespace are not controlled by an adversary. CRC32C is modeled as collision-free for the bytes encountered in an execution: a byte string different from every checksummed spelling actually encoded by the protocol does not validate merely through a checksum collision. Exact reconstruction is not excluded. If a torn later write supplies common constant bytes while older bytes supply the remainder of an earlier valid spelling, that spelling may validate and the protocol must handle it. The implementation uses CRC32C to detect tears and accidental corruption, not to authenticate data. Any safety claim is therefore probabilistic and conditional on this validity event and on the medium honoring completed barriers. This assumption does not exclude the exact old/new witness reconstruction in Limitation 1.*

2.3 Linux/ext4 refinement boundary

The transition system is not an ext4 implementation model. It is an application-visible contract that a backend must refine. On Linux, a successful `fdatasync` is used as the file barrier: Linux includes file size and other metadata required to retrieve the data in that operation, while an `fsync` of the file alone does not necessarily persist its directory entry. The latter requires an explicit parent-directory `fsync`.¹ A range-scoped `pwritev2(RWF_DSYNC)` provides the per-write `O_DSYNC` obligation for the addressed range; it is not `RWF_ATOMIC` and supplies no torn-write protection. A conforming backend may use that result only after the complete logical write has finished under the same durability obligation. A short completion must be continued, and an unsupported flag or any other error is an indeterminate failure rather than permission to fall back to unsynchronized success.

The initial conformance profile is a local, buffered, non-DAX regular file on ext4 in `data=ordered` (the default) or `data=journal` mode, beneath case-sensitive directories, with normal JBD2 replay enabled and a storage stack that honors cache flushes and write barriers. Ext4 casfold directories are outside this initial profile because distinct partition spellings can otherwise resolve to one namespace object. Files protected by `fsencrypt` or inline encryp-

¹Linux `fsync(2)`: <https://man7.org/linux/man-pages/man2/fsync.2.html>.

tion, and any other transforming storage layer whose tear granularity can produce application-visible bytes outside the old-or-issued envelope, also require a separate refinement argument.² UNO does not rely on ext4’s helpful pre-sync ordering: before a successful barrier, Assumption 1 permits any addressed-byte subset. Delayed allocation, block allocation, unwritten-extent conversion, and inode-size updates are considered unproved until the file barrier succeeds. A sync error is indeterminate and invokes Assumption 5.

The crash image in this paper means the logical regular-file and namespace state after normal filesystem journal replay. Mounting with journal replay disabled, using `data=writeback` or an unsafe `nobarrier` stack without a separate refinement argument, a device that lies about flush completion, misdirected writes, bit rot, and damage to bytes the protocol did not address are outside the conditional safety envelope. Independent byte survival is conservative relative to ordinary block-sized tears, but the “old or issued value, no neighboring damage” envelope remains an explicit device assumption rather than a guarantee obtained from ext4 journaling.

Assumption 3 (Exclusive namespace). *No external process renames, removes, replaces, or mutates participant files while the storage instance operates. Distinct simultaneously live logical keys name distinct device/inode identities; each participant name is a direct regular-file directory entry, not a symbolic link; and the owned namespace contains no hard-link aliases. A path becomes missing during a batch only through UNO’s deletion algorithm. Within a process, the namespace and participant locks serialize conflicting operations. A conforming implementation either preserves this property from a namespace created through its own API or detects and rejects aliases and symlinks before they can participate.*

Assumption 4 (Durable participant admission). *A participant is batch-eligible only after its immutable container header/incarnation, root page, exact selected predecessor authority, file length through that authority’s payload end, and every selected payload byte have crossed a file durability barrier. Its exact path-to-inode directory entry must then cross the required directory barrier. For create or atomic recreation, file contents are synced before the directory entry is declared eligible. Admission therefore always has a complete durable old vector and cannot name an inode whose path may disappear merely because creation was not durable. Group admission additionally rejects any repeated device/inode identity among its participants and any participant reached through a symbolic link.*

Assumption 5 (Fail-stop mutation errors). *After an uncertain mutable I/O error or cancellation following*

²Linux ext4 journal, mount-option, data-mode, and case-fold documentation: <https://docs.kernel.org/filesystems/ext4/journal.html> and <https://docs.kernel.org/admin-guide/ext4.html>. Linux fscrypt documents that file contents are encrypted in independent data units and that authenticated encryption is not currently supported: <https://docs.kernel.org/filesystems/fscrypt.html>.

physical-I/O admission, the affected open generation is poisoned and not reused. Recovery begins from disk. A caller does not continue mutating an instance whose durable frontier is unknown.

Assumption 6 (Fresh identities). *Incarnations and 128-bit group identifiers do not collide during the lifetime of reachable records. This is a probabilistic engineering assumption, not a cryptographic authentication guarantee.*

3 Physical representation

3.1 Blob geometry

UNO is a wrapper over ordinary blob I/O. The filesystem container supplies one immutable 4 KiB typed header containing the 128-bit incarnation. The wrapper reserves the backing blob’s first 4 KiB as two alternating 2 KiB slots; payload begins at physical offset 8192.

All root metadata uses the virtual filesystem coordinates in Figure 1. Selecting encoded payload length L therefore requires physical file length $8192 + L$. At the ordinary backing-Blob boundary, the immutable 4 KiB container header is hidden, so the corresponding logical length is $4096 + L$ and payload offset o maps to backing offset $4096 + o$. Every truncate in this paper means this translated full-file length, never the bare payload length L ; it cannot remove either the container header or root page.

Generation g occupies slot $g \bmod 2$. Consequently, writing generation $g + 1$ cannot overwrite the slot that contains generation g .

Invariant 1 (Consecutive nonwrapping slot lineage). *Every admitted transition from selected generation g proposes exactly $h = g + 1 < 2^{64}$ and targets slot $h \bmod 2$. Before the first byte of that transition is issued, the target slot’s only barrier-established source image is the all-zero bootstrap image when $h \leq 2$, or a canonical image of exactly generation $h - 2$. An attempt whose rejection completes consumes h with an exact abort authority rather than reusing it; provisional rejection enables no reuse. No path skips a generation or wraps the counter. A blob at generation $2^{64} - 1$ is read-only until migrated to a future format.*

3.2 Root header

Definition 2 (Canonical empty predecessor). *An all-zero 112-byte root-header region in slot zero makes Z_0 available as the fallback: generation zero, zero encoded length, unbound integrity, zero tag, and empty payload. It is an implicit predecessor, not an encoded root header, witness, or group certificate. Ordinary recovery selects it only when neither slot contains a recoverable ordinary root and no valid later-generation or wrong-parity root makes zero fallback ambiguous. The first candidate is generation one in slot one, so arbitrary rejected first-generation bytes in slot one do not alter the untouched Z_0 authority in slot zero.*

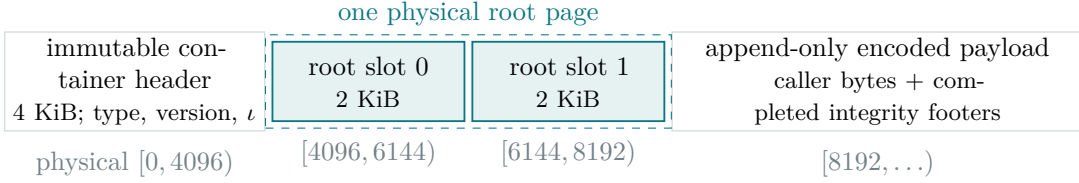


Figure 1: Filesystem geometry. At the generic wrapper boundary, backing logical offset zero is physical offset 4096, so the root page and payload are accessed only through ordinary `Blob` methods.

Every nonzero slot spelling begins with the 112-byte root header in Table 1. Unused bytes remain available for a batch witness. Integers are big-endian.

Let

$$\Sigma = \{P, B, R, A, M, T\}$$

and fix

$$(\sigma(P), \sigma(B), \sigma(R), \sigma(A), \sigma(M), \sigma(T)) = (0, 1, 2, 3, 4, 5).$$

For nonzero generation g and state $s \in \Sigma$, define the one-byte transition guard

$$\chi(g, s) = 1 + 6(g \bmod 3) + \sigma(s).$$

The root spelling is the seven ASCII bytes `CWUN014` followed by $\chi(g, s)$. Thus the state and the generation color occupy one addressed byte; they are deliberately not two separate fields that a torn write could splice. Assumption 1 chooses that byte as old or newly issued, not as independently torn bits. A decoder cross-checks the color against the encoded 64-bit generation before checking the root CRC32C.

Let D_{root} be the ASCII checksum-domain string in Table 1. The proposed R14 fixture stores, in big-endian order, $\text{CRC32C}(D_{\text{root}} \parallel \text{header}[0..108])$ at offsets 108–111. There is no intervening length or separator. This pins the representative byte vectors used by the executable model; it does not describe the checked-in R13 root codec as R14.

The root states are:

Symbol	State	Ordinary visibility
R	direct committed root	visible independent root
P	direct prepare	invisible direct prepare
B	batch prepare	invisible batch prepare
A	rejected-generation authority	visible predecessor-equivalent root
M	retained final root	visible independent retained root
T	removed final root	namespace deletion marker

The remainder calls the batch-prepared state **B**; earlier diagrams labeled the same abstract state “P.” Ordinary root recovery recognizes Z_0 , canonical independent R, A, and M. P and B are deliberately invisible; T is interpreted by namespace recovery rather than exposed as a blob.

Invariant 2 (Generation-colored provenance). *Suppose generation h is being introduced into one slot. Before a decision, the only generation- h guard bytes that the protocol may issue to that slot are $\chi(h, B)$ (or $\chi(h, P)$ for a direct publication) and, during consumption after provisional rejection, $\chi(h, A)$. It never issues $\chi(h, R)$, $\chi(h, M)$, or $\chi(h, T)$ before their respective commit guards. By Invariant 1, every byte already present from the preceding use of that slot has color $(h-2) \bmod 3$, which differs from $h \bmod 3$. Therefore an arbitrary subset of old, prepared, and abort writes cannot contain the guard byte required by an unissued committed or final root.*

Invariant 3 (Abort code-last publication). *The write that can place $\chi(h, A)$ is issued only after a successful barrier has made every other byte of the exact predecessor-equivalent $A_h(O)$ header durable. Until a later authority is durable, any exact A_h selected by recovery therefore has logical projection O ; no torn abort may validate with candidate metadata. A failed or canceled first or second phase returns no mutable handle and admits no later slot or payload reuse.*

Before the guard barrier establishes exact A_h , the abort body has no rollback authority. After every crash in either phase, recovery first decodes the entire resulting disk image again. If the surviving body bytes complete valid candidate evidence, recovery follows the decision path rather than resuming rollback. Exact A_h has precedence over the stale same-slot witness and is the first disk state that makes the old classification permanent.

3.3 Witness wrapper and local link

A B slot appends a 16-byte wrapper immediately after the root: 8-byte magic `CWUNOW13`, 4-byte link length, and a 4-byte CRC32C over the wrapper prefix and link under domain `_COMMONWARE_RUNTIME_ATOMIC_BATCH_WITNESS`. Up to 1920 link bytes fit in the slot.

Every prepare that introduces a generation writes one canonical complete 2 KiB slot image. A direct prepare is *P* followed by zeros. A batch prepare is *B*, its exact wrapper and link, then zeros through the end of the slot. Direct finalization changes the header to *R* and leaves the zero suffix; group finalization changes only the header to *M/T* and preserves the exact witness and zero padding. Completed rejection changes only the header to exact *A* and leaves any stale witness physically present but semantically suppressed. This full-slot prepare grammar is

Offset	Bytes	Field	Meaning
0	8	spelling	seven-byte R14 prefix plus one generation-colored state byte
8	8	generation	nonzero monotonically increasing generation
16	8	encoded length	selected payload length, including completed integrity footers
24	8	unfinished start	start of the only unfinished integrity unit
32	4	unfinished CRC32C	rolling CRC of the unfinished unit; zero when closed
36	4	integrity scheme	0 unbound, 1 variable, 2 fixed-size chunked
40	4	chunk width	nonzero only for fixed-size chunked integrity
44	64	application tag	caller-owned state published with this root
108	4	root CRC32C	domain-separated CRC over bytes 0–107

Table 1: The root header. The checksum domain is `_COMMONWARE_RUNTIME_ATOMIC_LOG_ROOT`.

recovery metadata: bytes outside a shorter write are not silently treated as harmless padding.

Each local link stores the fields in Table 2. The containing filesystem path supplies the local key; only the successor key is encoded. Thus every path appears once around the ring, not n times in a global descriptor.

Limitation 1 (Exact-witness provenance blocker). The generation-colored root guard proves that a state byte for generation h was issued; it does not prove that an adjacent generation-independent `CWUNOW13` wrapper was issued with that root. The wrapper CRC proves byte consistency, not issuance provenance. Under Assumption 1, checksum bytes may themselves be selected independently from the old and new slot images.

A concrete protocol-reachable trace starts with an independently finalized generation-one two-member group $H = \{a, b\}$, selects generation-two ordinary R predecessors, and admits a generation-three three-member append group $G = \{a, b, c\}$ into the same-parity slots. A crash before any G prepare barrier may retain exact issued B_3 roots and new candidate fields at a and b while retaining H 's group identifier, count, ordinals, and $a \leftrightarrow b$ successor fields. The wrapper checksums can also be exact old/new mosaics: in one independently reconstructed fixture, the old/new/hybrid CRC32C values are `47ad25a0/c3bcb36c/47ad256c` at a and `73a8b16e/47775580/7377b16e` at b . Each hybrid checksum byte comes from the old or issued-new checksum at that address and equals the canonical checksum of the hybrid link. This is exact reconstruction, not a CRC collision. Retaining no generation-three update at c then lets recovery accept a synthetic exact two-member ring, publish a and b , and leave c old. The observation of the admitted G is mixed.

Write `IssuedWitness( $G$ )` for the missing premise that every witness accepted as authority for G is byte-for-byte the canonical witness issued by the same prepare event and participant set as its candidate-generation guard. The current one-phase transition system does not establish this premise. The results below that relate a decoded ring to the originally admitted G are conditional on it.

Three repairs have materially different contracts. A two-phase, code-last batch prepare can first make the

candidate payload/length obligation and all 2,047 non-guard bytes of the exact 2 KiB slot durable, then issue and synchronize only $\chi(h, B)$. This preserves the existing noncryptographic fault assumptions but adds a second ordered durability layer. A one-layer design instead needs a root-to-witness commitment over every decision-relevant topology byte and an explicit computational *splice-binding* assumption; CRC32C or ordinary collision resistance is insufficient. If neither change is accepted, the admitted-but-unacknowledged old-or-new claim must be withdrawn. This architecture choice is unresolved here.

Definition 3 (Candidate). *A candidate c_i is the tuple*

$$(g_i - 1, o_i, B_i, R_i, a_i, \ell_i, z_i),$$

where g_i is the proposed generation, o_i its parity slot, B_i and R_i are the exact **B** and **R** headers, and (a_i, ℓ_i, z_i) describes the speculative payload suffix. The two headers must agree in every field except spelling and root checksum.

Definition 4 (Canonical integrity geometry). *For root fields (L, u, c, s, w) and encoded payload $V[0..L)$, write `IntegrityOK( $L, u, c, s, w, V$ )` when all of the following hold:*

1. $0 \leq u \leq L$; the unfinished bytes are exactly $V[u..L)$; c is their rolling CRC32C, with the canonical value zero when $u = L$.
2. Unbound is encoded only as $(s, w) = (0, 0)$ and requires $u = 0$. It has no completed units.
3. Variable integrity is encoded only as $(s, w) = (1, 0)$. The prefix $V[0..u)$ is a concatenation of caller-delimited nonempty data units, each followed by its four-byte big-endian CRC32C footer; $V[u..L)$ is the one possibly empty unfinished unit.
4. Fixed integrity is encoded as $(s, w) = (2, N)$ for $1 \leq N \leq 2^{32} - 1$. The prefix length u is a multiple of $N + 4$ and consists of exact N -byte data units followed by four-byte big-endian CRC32C footers; its unfinished tail satisfies $L - u < N$.

Ordinary root open checks the numeric scheme, bound, alignment, and closed-checksum conditions without scanning completed payload. Footer and rolling-CRC equality are construction invariants and are validated when the corresponding integrity unit or tail is read.

Field	Bytes	Constraint
link magic	8	CWUNOL14
group identifier	16	fresh γ
participant count, ordinal, flags	12	nonzero $n \leq 2^{32} - 1$; $0 \leq i < n$; only removal bit defined
local incarnation	16	equals immutable header incarnation
base generation, candidate slot offset	16	candidate is generation base+1 in the parity slot
exact B and R headers	224	two complete 112-byte candidate spellings
payload validation descriptor	20	offset, length, CRC32C; zero length means preflushed/no suffix
successor partition length + bytes	$4 + q' $	valid partition string
successor name length + bytes	$4 + x' $	raw name bytes
successor incarnation	16	exact physical generation expected at the successor path
fixed portion	336	leaves $1920 - 336 = 1584$ bytes for one successor partition/name pair

Table 2: Local successor link. Names are bounded per edge; they are never summed across the group into one descriptor.

Definition 5 (Eligible candidate). *Eligible(c_i) holds only if the candidate has generation exactly one above its base, does not wrap, occupies the parity-selected slot, satisfies Invariant 1’s exact same-slot source condition, has a canonical root encoding, and would pass ordinary root recovery at the candidate encoded length. IntegrityOK holds and the issued physical payload covers that encoded length. Every candidate payload byte must be exactly one of:*

1. an unchanged byte selected by the predecessor root;
2. a current stamped value covered by preflush durability credit; or
3. a current stamped value in the candidate’s one exact contiguous speculative suffix, whose descriptor ends at the candidate length and carries its CRC32C.

A prepare transition may issue no candidate for which this predicate or the aggregate suffix bound is false.

Definition 6 (Allowed torn transition). *Let X_i be the 112 bytes currently read from candidate slot o_i . For a retained participant,*

$$\mathcal{S}_{i,j} = \{B_i[j], R_i[j], A_i[j], M_i[j]\},$$

$$\text{Transition}(X_i, c_i) \iff \forall j, X_i[j] \in \mathcal{S}_{i,j}.$$

For a removed participant, $T_i[j]$ is also admitted. Here A_i is the predecessor-equivalent abort root, while M_i and T_i are deterministically derived from R_i by changing the generation-colored spelling and recomputing the root checksum. Byte membership alone is not a decision predicate: recovery also requires the guard byte to name a state whose write was enabled by the transition system. Invariant 2 makes that issuance fact disk-observable.

Recovery applies exact- A_i precedence before using this predicate as candidate evidence. Exact A_i is included in the allowed byte set so interrupted repair images can be classified, but an exact abort root suppresses its same-slot witness and cannot participate in a deciding ring. For a non- A image, the wrapper and link must themselves validate exactly. Installation overwrites only the first 112 bytes, so the already durable witness remains available even when the final root write tears. The appearance of

R_i in Transition makes its byte values legal during repair; it does not make an ordinary root proof of this group.

Definition 7 (Group-bound final certificate). *A final certificate for participant i of G is the pair consisting of (a) the exact candidate M_i spelling for a retained participant or T_i for a removed participant and (b) the adjacent exact wrapped witness naming G , i , and c_i . An exact R_i is not a group certificate because the root header contains no group identifier.*

Definition 8 (Abort authority). *Let authority at generation g select logical state O_i , and let its opposite slot contain a rejected candidate or an orphaned partial transition for generation $h = g+1$. The abort authority $A_i = A_h(O_i)$ is a canonical 112-byte root header with guard $\chi(h, A)$ and with every logical field copied from O_i . It consumes generation h without changing the logical projection. Recovery gives an exact A_i precedence over any same-slot witness; the stale witness may remain physically present but has no authority.*

Abort publication is code-last. First, while leaving the one-byte spelling guard unaddressed, recovery writes the other 111 root bytes of A_i —including the checksum computed for the final A_i spelling—and crosses a file barrier. Then it writes only $\chi(h, A)$ and crosses a second barrier. Before the first barrier, no A guard exists. After it, every non-guard byte is the exact predecessor-equivalent A_i value. A crash during the one-byte second phase therefore leaves either the prior non- A guard or the complete exact A_i ; it cannot combine an A guard with a candidate-valued body. On the live path, an abort authority is established only after the second barrier succeeds. If that barrier is interrupted, the open generation is poisoned; after a crash, recovery may accept a complete exact A_i that happened to land, or repeat the two phases from the partial image. It never relies on the pre-crash process having observed success.

Definition 9 (Canonical and quiescent slot). *A complete slot image is canonical exactly when it is one of: (a) the all-zero unused image; (b) an exact P_h or R_h header followed only by zeros; (c) an exact B_h , M_h , or T_h header*

followed by its exact same-generation wrapped link and zeros after the declared link; or (d) an exact A_h header, whose ignored suffix may retain the rejected witness. After recovery has selected authority at generation g , its opposite-parity slot is g -quiescent only when it is all zero or is a canonical image of generation $h < g$. Every other opposite-slot byte string is an orphaned partial transition, even if no root or wrapper inside it validates. An exact canonical image with generation at least g is resolved before this predicate is evaluated; it is not classified as inactive.

4 Logical protocol state

4.1 Per-blob state

The selected protocol state of one blob is

$$S = (g, L, u, c, s, w, t, V[0..L]),$$

comprising internal generation, encoded length, unfinished-integrity start and CRC, integrity scheme and width, 64-byte caller tag, and the exact encoded payload bytes selected below L . Its logical projection is

$$\pi(S) = (L, u, c, s, w, t, V[0..L]),$$

which omits the protocol generation. Payload content is part of the logical value even though it is not repeated in the root. The in-memory state may additionally contain an unpublished append tail or a shorter staged length.

For $g = 0$, S is the canonical Z_0 state of Definition 2; for $g > 0$, it is selected by a checksummed root spelling.

For proof purposes each issued payload byte carries a monotonically fresh write stamp. A durability credit is a triple (o, v, e) saying that a successful file barrier covered value v written at offset o with stamp e . Credit belongs to that exact value and stamp, not merely to the offset. Rewind invalidates every credit at or above the new physical end before those offsets may be reused; replacement appends receive new stamps. A concurrent preflush credits only the snapshot of issued stamps ordered before its barrier. This is the abstract counterpart of the implementation's payload checksum/frontier tracker.

Appends issue payload writes immediately; UNO does not retain the complete epoch in memory. A rewind below the selected length fences later appends until the shorter root is durable, after which the backing file may be truncated in place. If a crash loses that physical truncate, root recovery repeats it before the discarded offsets are reused. Regardless of whether the physical truncate survived, replacement bytes remain uncovered until their own exact stamps are synced or included in the candidate's validated suffix.

Protocol-owned payload integrity is orthogonal to batch publication. A completed caller-delimited or fixed-size unit has one 4-byte CRC32C footer. The selected root stores the start and rolling CRC of the only unfinished unit. Ordinary independent-root recovery reads neither

completed payload nor the unfinished unit; validation occurs only when that unit is requested. Namespace recovery of an unresolved group may additionally stream its bounded speculative suffix before choosing a root.

4.2 Groups and the canonical ring

After duplicate elimination and conflict checks, participants are sorted strictly by key:

$$\text{key}(p_0) < \text{key}(p_1) < \dots < \text{key}(p_{n-1}).$$

Link i declares ordinal i , count n , and successor $p_{(i+1) \bmod n}$. Every link carries the same fresh group identifier γ . Removal is a local bit and is also reconstructed as a sorted subset of participants.

The same canonical order is used for participant locking, ordinals, successor validation, and descending deletion. Caller order therefore cannot create a distinct ring or opposing lock order.

Invariant 4 (Exact closed ring). *A ring is complete only if following successors for exactly n steps finds: one unique matching witness at each exact key/incarnation; the same γ and n ; consecutive ordinals modulo n ; strictly increasing keys after rotation to ordinal zero; pairwise-distinct current device/inode identities; and closure to the starting exact incarnation on step n , neither earlier nor later.*

5 Transition system

A configuration contains the namespace map, each inode's two raw slots and payload bytes, the durable base and post-barrier updates from Assumption 1, and volatile lock/carried-group state. An initial configuration contains only durably and directly named regular-file participants whose two slots are canonical, whose selected roots or canonical Z_0 predecessors and payloads pass ordinary recovery, and whose logical-key-to-inode map is injective with canonical integrity geometry; it contains no unsuppressed prepared witness. A configuration is *protocol-reachable* iff it follows from an initial configuration by the guarded transitions below, an allowed crash, and any prefix of recovery. These rules are normative: a backend that can produce another transition does not refine this protocol.

Stage payload.

Append writes encoded bytes once at their final offsets without changing a selected root. Preflush may establish stamp-specific durability credit. A rewind below the selected length must become a selected durable root before an append may reuse discarded offsets; it invalidates all old credit for those offsets. No offset named by a rejected candidate may be truncated, overwritten, or reused while that candidate could still be rediscovered as an exact ring; the abort-authority transition below must run first.

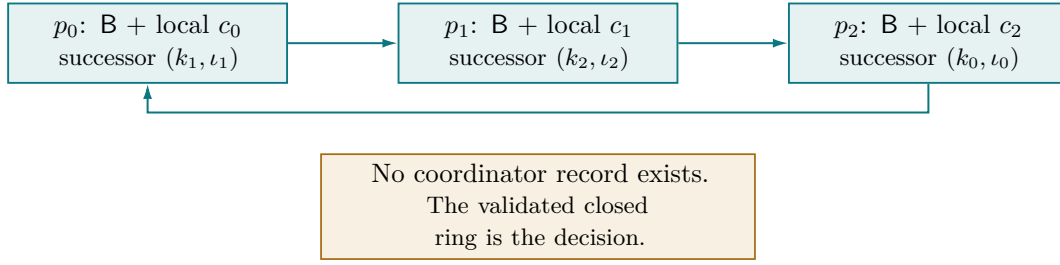


Figure 2: Three participants linked in canonical order, closing on the exact starting incarnation. A one-participant publication self-links.

Prepare G .

While all keys in K_G are exclusively locked, validate exact current directly named regular files, incarnations, and pairwise-distinct current device/inode identities, construct only Eligible candidates, and establish Invariant 5. Issuing the first B_i /witness byte is $\text{admit}(G)$. Each participant receives its exact canonical full-slot B_i /wrapped-link/zero-padding image in its candidate slot. No prepared root is an ordinary visible root. This is the one-phase prepare analyzed by Limitation 1; without an additional issuance mechanism, a checksummed ring recovered from a torn prepare need not be the ring this transition is sued.

Normal decision and acknowledgement.

Submit each participant's durability obligation. Only the successful join over *all* obligations establishes $\text{decide}(G)$ on the live path; only then may $\text{ack}(G)$ occur. The obligations have no protocol order among them.

Recovery decision.

If no live join exists, recovery may establish $\text{decide}(G)$ only after excluding exact A at every candidate generation, then reading one exact closed ring and checking every $\text{Transition}(X_i, c_i)$ and $\text{Eligible}(c_i)$, and validating all non-preflushed payload suffixes. A group-bound final certificate proves that this validation or the normal join occurred earlier only when its exact witness chain reaches every participant that is not already independently final. A broken ring is accepted without traversal only after independent-final roots or a serialized later authority already carry the proof. A missing path, incomplete link, ordinary R_i , or malformed candidate is never a prepare vote.

Provisional rejection and consumption.

If the current disk image does not prove a candidate reached through an exact wrapped witness, recovery first resolves the predecessor authority. If that predecessor is a carried linked decision, recovery makes it group-wide independently final before suppressing any of its links. Recovery then attempts to publish A_i from Definition 8 code-last into one required witness slot: synchronize the 111 non-guard bytes, then synchronize the one-byte A guard. The initial rejection is not durable authority. After any crash or

uncertain barrier, recovery re-reads every root, witness, and required payload byte before continuing. If the surviving abort-body bytes now complete valid candidate evidence, it establishes $\text{decide}(G)$ and finalizes new. If exact A_i is present, it suppresses the stale witness and selects old. Only after predecessor finalization and exact A_i may recovery return a mutable handle or permit a candidate payload offset to be truncated, overwritten, or reused. The exact abort root consumes the candidate generation while preserving the old logical value; its generation-colored guard is distinct from every unissued $R/M/T$ guard for that generation.

Normalize an orphaned slot.

After recovery resolves every exact higher-generation root, final certificate, and wrapped candidate, it evaluates Definition 9 from the selected generation g . If the opposite slot is not g -quiescent, recovery treats it as an unrecognizable partial transition. It first makes any linked selected authority group-wide independently final, then publishes A_{g+1} code-last into the opposite slot. No wrapper is required to enter this branch. Recovery may return a mutable handle, truncate or reuse payload, or use the other slot for the next generation only after the abort-root barrier succeeds. A crash during normalization restarts the complete byte-derived classification, including candidate and payload evidence that a partial abort body may have completed; it never relies on memory that the abort write had once begun.

Finalize.

Only after $\text{decide}(G)$ may recovery or normal execution issue M_i for retained participants and T_i for removals. Each repair attempt is an ordinary fallible write followed by a file barrier and may be interrupted at any byte subset. The next recovery reclassifies that partial image before retrying. Group-wide independent finalization holds only after *every* participant's final write has succeeded or an exact final image has been recovered from a crash; until then all original witnesses remain intact, so a reachable exact certificate or the complete ring can repair torn peers.

Unlink.

No participant path may be unlinked before group-wide independent finalization. Cleanup may target

only a removed participant after revalidating its exact key, incarnation, witness, tombstone, and currently named inode. It then synchronizes the parent directory. A missing or replaced path is left untouched.

Proof transfer and slot reuse.

A witness required to recover G may be overwritten, suppressed, or made unreachable only after either (a) group-wide independent finalization of G , or (b) a complete decision for a consecutive group over exactly the same ordered keys/incarnations has become durable in the alternate slots. Case (b) transfers the serial prefix to the successor ring; if the successor is incomplete, the predecessor ring remains intact. Any other overlapping operation—including an abort root that would suppress a linked predecessor—first takes route (a). Before a direct $P \rightarrow R$ publisher may prepare a slot containing rejected evidence, recovery must consume that rejected generation with the same abort authority; the direct publication then uses the next parity slot. Merely invalidating a few wrapper bytes and reusing the rejected generation is outside the transition system. An ordinary R is never group evidence.

Conflicting non-group mutation.

Create, direct publish/rewind, exact remove, and whole-partition remove acquire the same logical-key/namespace exclusion as groups. Before their admission, every decided carried group touching a target key is made group-wide independently final; these operations are not exact-ring successor transfers. Create first resolves any tombstone, then installs a fresh named inode generation and satisfies Assumption 4 before returning a batch-eligible handle. Direct publication uses its own invisible prepare and durable independent root, including the proof-transfer and abort-authority guards above. Remove first resolves touching witnesses, then unlinks only the exact current target, synchronizes every affected directory, and invalidates its live generation while retaining namespace exclusion. A whole-partition remove applies that rule to every contained key. A crash before admission leaves selected states unchanged; after admission, the operation follows its ordinary API recovery semantics and every surviving effect is serialized after the finalized groups it touched. UNO makes no new atomicity claim for whole-partition removal itself.

Recovery selection.

Candidate rings are considered by descending generation. Any newer exact independent R , M , or T state at a key suppresses older witnesses at that key; T maps the key to logical absence. An exact local group-bound M/T certificate is selected before ring traversal and is never rolled back merely because a deletion has broken its successors. For the newest remaining prepared candidate, recovery either rejects an incomplete ring with no certificate provisionally

and begins abort publication, or establishes the decision and finalizes the whole group. Before a provisionally rejected candidate’s predecessor becomes mutable, an exact code-last A has made the old classification durable. If neither an exact candidate nor a newer authority is recognizable, the orphan-normalization transition resolves every non-quiescent opposite slot before mutability. It never exposes B directly. Recovery transitions obey the same finalization, unlink, and proof-transfer guards and may themselves crash.

The rules separate an evidence predicate from a historical event. In particular, a final certificate proves $\text{decide}(G)$, not necessarily that the original pre-crash sync calls returned: recovery itself may first validate a fully surviving unacknowledged ring and then issue the certificate.

6 Normal execution

6.1 Preparation

The namespace owner performs the following steps while conflicting namespace and participant mutations are excluded.

Listing 1: Prepare one group G .

```
canonicalize operations by (partition, raw name)
validate current handles, exact incarnations, rewinds,
    and names
reject symbolic-link entries
reject repeated device/inode identities under distinct
    keys
for each dirty participant i:
    derive candidate (B_i, R_i) for generation g_i + 1
    choose the bounded payload suffix not already durable
    if suffix is ineligible or exceeds its assigned
        recovery share:
            sync the current immutable payload; then use an
            empty suffix
require every candidate to satisfy Eligible(c_i)
choose fresh group id gamma
encode one canonical full slot: B_i || wrapper(link_i) ||
    zero padding
preflight every link against its 1,920-byte local
    capacity
```

Pending payload can be arbitrarily large subject to address-space, file, and implementation limits. Background preflush may run while appends continue. The current backends divide the 64 MiB recovery budget among locked mutations, which may preflush a large participant even when the aggregate suffix would fit. The protocol requires only the resulting aggregate bound. Preflush changes no visible root and never rewrites payload.

Invariant 5 (Payload coverage). *For every candidate whose ring can still establish or repair a decision (that is, it has not been durably consumed by A or suppressed by a newer authority), each payload byte below its suffix start either is an unchanged byte selected by the predecessor root or has stamp-specific durability credit from a successful file barrier before the B write was issued. The*

remaining exact contiguous suffix ends at the candidate length and is either empty or described by a CRC32C over its current stamped values. Across one group, nonempty suffix lengths sum to at most 64 MiB. Rewind and offset reuse cannot reuse credit belonging to an earlier value.

6.2 Decision round

For every retained participant, UNO writes B_i plus its wrapped local link and submits a file durability operation. A deletion candidate names the last committed length, not discarded pending bytes, and its prepared witness write is itself made durable. The per-file obligations may run concurrently through the runtime backend; the protocol does not depend on any particular kernel submission interface or on physical flush coalescing.

Listing 2: Establish and acknowledge a decision.

```
for every participant i, concurrently where possible:
  write candidate slot := B_i || wrapper(link_i)
  make that participant's prepare record durable
  wait until every participant durability obligation
    succeeds
  record the complete decision in process memory
  return Ok from start_apply      // durable
  acknowledgement
```

There is no later coordinator write. If acknowledgement occurred, all prepare records and covered payload are durable. The intended rule for an earlier crash was to choose old for an incomplete ring and new for a complete ring. Limitation 1 shows that this dichotomy is false for the current one-phase transition: old and issued-new slot bytes can reconstruct a complete ring over only a strict subset of the admitted participants.

Invariant 6 (Prepared invisibility). *No B root is selected by ordinary blob recovery. Until namespace recovery validates an entire group, each participant's preceding Z_0 , R, M, or complete predecessor-ring proof remains its only visible authority.*

Invariant 7 (Rejected-candidate generation consumption). *If recovery has completed rejection of a candidate group and has made its predecessor available for mutation, then any collective predecessor proof was made group-wide independently final before that first availability, and one required rejected-ring slot contains a durable exact A root at the rejected generation. Until exact A is established, candidate payload ranges remain immutable and no truncate or offset reuse may be admitted. By parity, the abort-root slot cannot be addressed again before a strictly higher-generation authority exists in the alternate slot, so future arbitrary-subset rewrites cannot restore the old group's authority.*

Invariant 8 (Mutable-return slot normalization). *Recovery returns a mutable handle selecting generation g only if the opposite slot is g -quiescent, or after a durable A_{g+1} has advanced the selected authority without changing its logical projection. A failed or canceled normalization re-*

turns no handle. This guard applies even when neither a root nor a wrapper in the opposite slot validates.

6.3 Materialization and activation

After the decision, a retained candidate can be rewritten as independently recoverable M; a removed candidate becomes a payload-preserving T. Each rewrite is a 112-byte write followed by a file barrier, and may itself be interrupted at any addressed-byte subset. A live repair counts as durable only after that barrier succeeds; a post-crash exact root is reclassified from disk. The exact M/T header together with its exact adjacent witness is a group-bound final certificate. An ordinary R is never such a certificate. Normal execution issues no final rewrite until every participant prepare obligation has succeeded; recovery may do so after independently validating a complete ring and its payload.

The certificate claim depends on Invariant 2, not merely on CRC validation. Before $\text{decide}(G)$, no transition writes $\chi(g_i, M)$ or $\chi(g_i, T)$ into candidate slot i . Old same-parity state has a different color, and the B/A state bytes for generation g_i are distinct. Therefore an exact M/T spelling cannot be assembled by a torn prepare or abort write before the guarded final write is issued.

Removal-free groups may remain in the linked state and be carried in process memory. A following group with exactly the same participant keys/incarnations and consecutive generations may use the other slots and supersede it directly; that successor may itself contain a removal and will then be eagerly finalized. A different participant set or any conflicting namespace operation first materializes the carried decision group-wide.

Invariant 9 (Proof transfer before reuse). *A witness needed to recover a decided group is not removed, overwritten, or suppressed unless either (a) every participant in that group has a durable independent final root, or (b) a consecutive group over the exact same participant ring has established a complete durable successor decision in the alternate slots.*

6.4 Deletion

Deletion uses the following normal completion sequence:

1. durably write M for every retained participant and T for every removed participant;
2. wait for *all* final-root writes to succeed;
3. visit removed participants in descending canonical ordinal;
4. for each one, verify exact key/incarnation, witness, tombstone, and current inode; unlink; then synchronize its parent directory before moving to the next lower ordinal.

Invariant 10 (Final-before-unlink). *No path is unlinked on behalf of group G until every participant of G has a durable independent final root.*

Define `IndependentFinalized( $G$ )` to mean that every retained participant has an exact durable M root, every removed participant has an exact durable T , and the durable local witness remains beside each spelling. Each record then determines that participant's new logical state without traversing the ring.

Invariant 11 (Independent authority before unlink). *If an unlink on behalf of G is enabled or has been issued, then `IndependentFinalized( $G$ )` was established before the first such unlink. Until admission of a later mutation touching a key in K_G , each retained participant remains independently M , while each removal is represented by its exact T or by the missing name produced from that tombstone. After such an admission, any replacement authority is produced by the serialized later transition and denotes a state ordered after G , never a state before G .*

Invariant 12 (Tombstone dominance). *Once namespace recovery identifies an exact current-incarnation T , that incarnation is logically absent and remains so unless a serialized create installs a fresh named generation. Exact-name open may report absence or perform that create, and scan must omit the tombstoned incarnation; neither path may fall through to its older R in the alternate slot.*

Invariant 13 (Incarnation-safe cleanup). *Cleanup may unlink a path only after validating the expected participant incarnation, exact T , wrapped witness, and currently named inode. A missing path or a different incarnation is never removed on behalf of the old decision.*

Descending ordinal order and a directory barrier after each unlink give the common crash path a cheap, resumable frontier. They are a reclamation policy, not a safety dependency after Invariant 11 holds. A later exact-name open may remove one surviving tombstone, and a partition scan may remove several and synchronize the directory once. Such out-of-order cleanup can break the ring and strand other physical tombstones, but it cannot change their logical state.

7 Recovery

Recovery has two layers. Namespace recovery first resolves linked decisions and deletions. Only then does ordinary per-blob recovery choose the newest valid R or M root, or the canonical Z_0 fallback, and truncate any physical surplus to the selected length.

7.1 Discovering a ring

Opening one atomic participant inspects its two witness wrappers. For each intact wrapper, recovery decodes the local link and follows exactly its declared count. At each successor it:

1. opens the exact path without following symlinks;
2. validates the immutable container header and required incarnation;

3. searches the two slots for exactly one witness matching group, count, expected ordinal, candidate slot, and local incarnation; and
4. verifies the candidate transition and exact wrapped witness.

Missing, duplicated, ambiguous, early-closing, out-of-order, stale-incarnation, overlong, malformed, or non-closing paths provisionally reject a prepared candidate only when the anchor lacks an exact group-bound final certificate. An exact local M/T certificate is examined before traversal because legitimate deletion may already have broken the ring. Recovery never scans unrelated names.

After all recognizable candidates and authorities have been resolved, recovery also inspects each complete raw slot image. If the inactive slot fails Definition 9, it cannot be silently ignored: it is normalized through the same abort authority before a mutable handle or payload reuse is permitted. This branch is deliberately independent of wrapper validity.

Listing 3: Recover from one intact witness.

```

if anchor slot contains exact A for this candidate
  generation:
    SelectPredecessorEquivalentAbort(anchor)
    return OLD // exact A suppresses
    the stale witness
if anchor has an exact group-bound final certificate M/T:
  links := TryFollowExactlyNLinks(anchor)
  if links form one exact canonical closed ring:
    FinishDecision(links) // repair any still-
    linked peers
  else:
    SelectIndependentFinal(anchor)
    TryOrderedDeleteCleanup(anchor)
    return FINAL
links := FollowExactlyNLinks(anchor)
if links do not form one exact canonical closed ring:
  IndependentlyFinalizeLinkedPredecessorIfAny()
  PublishAbortRoot(anchor, predecessor logical value)
  return OLD only after exact A is durable
if any peer has an exact group-bound final certificate M/
T:
  FinishDecision(links) // decision was already
  durable
else if every candidate is Eligible and every transition
and suffix validates:
  FinishDecision(links) // complete prepare
  survived
else:
  IndependentlyFinalizeLinkedPredecessorIfAny()
  PublishAbortRoot(anchor, predecessor logical value)
  return OLD only after exact A is durable
authority := select the preceding visible or carried
authority at generation g
if opposite raw slot is not g-quiescent:
  IndependentlyFinalizeLinkedAuthorityIfAny(authority)
  PublishAbortRoot(g + 1, logical_value(authority))
  restart recovery selection
return authority only after the mutable-return
normalization guard

```

The broken-ring certificate branch never rolls back or aborts the final anchor. Within protocol reachability, a ring can become broken after an exact local final certificate only after the final-before-unlink guard made every

participant independently final, or after a serialized later mutation. The anchor therefore selects its own new state while peers select their M/T , missing, or later authorities.

The abort root is required even when the provisional rejection was caused only by payload validation. Otherwise a later rollback and append could reuse offsets that the stale witness treats as preflushed, and a second crash could make the old suffix validate against a payload prefix that old group never proposed. Merely zeroing or complementing witness bytes is insufficient: successive partial slot rewrites could reconstruct an older byte pattern. The generation-consuming A root avoids that ABA shape without rewriting the witness. Making a linked predecessor independent first preserves its group-wide authority when the newer abort root suppresses its ring witness. The pre- A classification is deliberately non-monotone across crashes; exact A , not the failed validation, makes the old outcome monotone without retaining in-memory write stamps on disk.

A crash during abort publication does not create an undocumented fourth case. The next recovery first reruns the ordinary disk predicates. It may recognize a complete candidate ring—including one completed by surviving abort-body bytes—and decide new; recognize complete A and select old; or see a noncanonical raw slot and repeat normalization. If the torn image happens to equal an older canonical quiescent image, then no distinguishing byte of the rejected higher-generation image remains in the recovered durable base; it is safe to reuse under the validity event in Assumption 2. In particular, while introducing generation $g + 1$, the guard byte can come only from the older same-slot color, $\chi(g + 1, B)$, or $\chi(g + 1, A)$. It cannot equal an unissued $\chi(g + 1, R)$, $\chi(g + 1, M)$, or $\chi(g + 1, T)$. No branch depends on remembering that a pre-crash process intended to write A .

If both slots contain recoverable linked candidates, they are considered by descending generation. A later exact independent R , M , or T suppresses stale witnesses. This permits exact-ring supersession while preventing replay of an old group over a newer retained value or deletion. Same-generation R cannot certify a stale ring: it is either separated from batch use by the direct-publication retirement guard or is a protocol violation.

7.2 Arbitrary torn final roots

The final M or T write may tear at any subset of its 112 addresses. The root CRC may consequently be invalid. Recovery does not need to guess a prefix boundary: the exact candidate witness supplies all legal byte values, and **Transition** checks every byte independently. Once another reachable exact group-bound M/T certificate proves that installation began, recovery retries the deterministic complete final spelling. Every retry can tear again; only a successful barrier or a later recovered exact spelling advances that participant to independent final state.

This repair is safe only because the witness was made durable first and final installation writes do not overlap

it. If a medium may damage unwritten neighboring bytes, Assumption 1 does not hold.

7.3 Recovery with missing participants

A missing path has two fundamentally different meanings. Recovery must not conflate them.

No group-bound final certificate. If following a prepared ring encounters a missing path before any group-bound M/T certificate is known, the ring is incomplete and supplies no authority. Recovery provisionally rejects it. In particular, a missing blob whose local removal bit would have been set is *not* evidence that the batch committed. No remaining predecessor becomes mutable until the rejected generation is consumed by exact A on the required reachable witness authority. In a protocol-reachable execution the missing path cannot have been caused by UNO itself, because UNO issues no unlink before finalization; an actual pre-finalization missing path is external namespace interference or corruption and may fail closed rather than fabricate a complete old vector.

Deletion broke a finalized ring. After unlinks start, the full ring may no longer be traversable even though the decision is already durable. This is safe because the first unlink is downstream of independent final roots at every participant. Present M roots select new retained values, and present T roots independently denote logical absence. Before any later mutation of a key in K_G , a protocol-generated missing path can only be a removed participant: cleanup for G never unlinks a retained participant. A later operation may replace or remove a retained incarnation only after Invariant 9 has made G independently recoverable; that later operation is ordered after G and is outside Definition 1’s observation interval.

The implementation still attempts the following eager reclamation walk when ordinal zero survives:

Listing 4: Recover an ordered delete frontier from ordinal zero.

```
require exact final candidate and witness at ordinal 0
prefix := []
for expected ordinal j = 1 .. n-1:
  append current exact final link to prefix
  if exact successor path/incarnation is unavailable:
    dirsnc(parent(next))    // persist observed
  frontier
  for link in reverse(prefix):
    if link.removed:
      verify T; unlink; dirsnc(parent)
  return COMMITTED
require successor has exact M or T and expected link j
current := successor
reject a non-closing intact chain here; full-ring
recovery handles it
```

On the normal descending path, a missing exact successor or a path with a replacement incarnation ends forward traversal. After re-synchronizing that successor’s

parent directory, the implementation may reclaim the already verified lower prefix in reverse order. The exact final ordinal-zero certificate and group-wide independent roots authorize that cleanup; the hole or replacement is never itself a commit vote. A non-prefix hole therefore limits discovery but does not make verified prefix cleanup unsafe. Logical recovery relies on Invariants 11 and 12; if the parent directory itself is absent, the current availability deviation in §13 applies.

An exact-name open that finds a current-incarnation tombstone first reports the old incarnation absent, unlinks the lingering name, and may then create a fresh named inode generation. Atomic open mints a fresh V2 incarnation; ordinary open may create a non-atomic V1 blob. A partition scan applies the same test to every canonical file, unlinks every confirmed tombstone, then synchronizes the partition directory once. Consequently no surviving tombstone is returned as live even if the ring or the descending frontier is no longer traversable. Tombstones that are neither opened nor encountered by a scan may remain allocated indefinitely; this is delayed reclamation, not logical resurrection.

Limitation 2 (Externally missing retained participant). The missing-path argument relies on exclusive namespace ownership. UNO itself unlinks only exact tombstones, so a missing retained participant is not protocol-reachable after independent finalization. A file removed by another process or lost independently by the medium is corruption outside the conditional crash-atomicity target; UNO does not reconstruct its payload or promise failover semantics.

7.4 Recreation and stale paths

When a deleted key is recreated atomically, its new 128-bit incarnation differs. An ordinary V1 replacement has no matching V2 incarnation. Ring traversal therefore rejects either as the expected participant. Stale cleanup also compares the inode opened for validation with the inode currently named at the path immediately before unlink. Thus a replay of an old tombstone cannot remove a replacement that reuses the same human-readable name.

8 Safety argument

All results in this section are conditional on Assumptions 1–6. “Valid” means checksums do not collide. The invariants are consequences of the guarded transition system rather than additional assumptions. Statements that identify a decoded ring or certificate with the originally admitted group additionally require $\text{IssuedWitness}(G)$ from Limitation 1. The current one-phase prepare does not establish that premise, so the crash-atomicity result is a conditional conjecture rather than a theorem of the specified transition system.

Lemma 1 (Invariant preservation). *Every protocol-reachable configuration satisfies Invariants 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, and 13.*

Proof. Induct on transitions from the initial configuration. Stage-payload steps do not change a selected root; their stamp rule invalidates credit on rewind/reuse and grants credit only after a barrier. Prepare admits only eligible candidates, constructs one canonical exact ring, and establishes the payload bound before issuing any B byte. Initial-state injectivity, alias rejection, conforming creation, and exclusive namespace ownership preserve distinct participant inodes. A crash can retain arbitrary addressed bytes but cannot invent an unissued generation-colored state byte or alter the alternate slot; B remains invisible by the recovery selection rule. Reject-and-consume first applies the ordinary group-wide finalization guard to any linked predecessor, then establishes all non-guard A_i bytes before issuing its state guard. Thus an exact A_i has the predecessor projection even if either phase crashes. Parity leaves A_i untouched until a higher-generation authority exists in the alternate slot, so later partial rewrites cannot make the consumed generation selectable again. Before exact A_i , however, the classification remains provisional. A crash during abort publication returns no handle; canonical slot decoding first re-evaluates complete candidate and payload evidence, then either decides new, selects exact A_i as old, or classifies the raw opposite slot as non-quiescent and retries normalization. The same byte-derived guard covers a partial transition whose wrapper no longer validates. Establishing a decision changes only a monotone fact; failure to establish it does not. Finalize is guarded by the positive decision fact, preserves the non-overlapping exact witness, and reaches group-wide independence only after every M/T write succeeds. Unlink is guarded by group-wide independence and exact incarnation/witness/inode checks. Proof transfer permits witness retirement only after group-wide independence or a complete same-ring successor decision in the alternate slots. Recovery uses the same guards, prioritizes newer $R/M/T$ authority, treats every surviving exact T as absence, and may crash back into the same induction. These are all transitions in Section 5. $\square$

Lemma 2 (Alternating-slot predecessor preservation). *Issuing candidate generation $g + 1$ cannot overwrite the slot record for generation g .*

Proof. Slots are selected by generation parity. Consecutive generations have opposite parity. Under the addressed-write model, the candidate write can alter only its own 2 KiB slot, so the prior slot is unchanged. For $g = 0$, generation one uses slot one and leaves slot zero’s all-zero root-header fallback. For $g > 0$, the prior record is either an independent root or the complete linked proof carried from the immediately preceding exact-ring group. $\square$

Lemma 3 (Abort-authority permanence). *Once A_i for rejected group G is durable, no later protocol-reachable crash image can use participant i as the exact witness required to select G .*

Proof. Immediately after the barrier, the candidate slot contains a valid independent root at G 's local generation whose logical projection is the predecessor. The old witness bytes may remain, but same-slot A has recovery precedence over them. Payload writes do not address the slot. By alternating parity, the next root or witness update for this participant uses the other slot. Before any protocol transition may reuse the abort-root slot, that intervening generation has become an independent root or a complete durable linked decision; it is strictly newer than G and suppresses G under recovery selection. This suppression is established before the reused-slot write is issued, so it survives every arbitrary subset of that write and every later crash. Thus bitwise mixtures may reconstruct old bytes physically, but they cannot restore G 's authority. $\square$

Lemma 4 (Disk-derived orphan normalization). *A crash during candidate preparation, candidate rejection, or abort-root publication cannot make recovery return the predecessor for mutation while an unrecognized higher-generation slot remains reusable.*

Proof. Recovery first resolves every exact root, certificate, and wrapped candidate. If none applies, it compares the complete opposite slot against the canonical grammar in Definition 9. A noncanonical image enables no mutable return and is overwritten by another code-last A_{g+1} attempt. A crash during that attempt restarts this classification; if the new image supplies complete candidate evidence, recovery decides new rather than assuming the earlier rejection remains valid. During the first phase the A guard is absent; during its second phase all other A bytes are already barrier-established. Hence an exact A_{g+1} is a newer independent authority with the same logical projection. An exact higher candidate is processed normally. An all-zero or strict lower-generation canonical image contains no unsuppressed higher-generation protocol record in the recovered durable base. If generation $g + 1$ remains distinguishable, Invariant 2 limits its guard byte to the prepared/direct-prepare state or the abort state until a commit guard actually issues another state. Thus no torn normalization can synthesize a new visible or final authority. Every path that permits mutation has either removed the orphan from the durable base or placed it behind a newer authority. Repeating the argument after each crash establishes the claim without volatile intent state. $\square$

Lemma 5 (Incomplete prepare is invisible). *If a candidate ring is missing any exact link, eligible candidate transition, or required payload byte, and no group-bound final certificate exists, recovery does not expose any candidate state from that ring.*

Proof. Ring traversal requires all n exact participants and closure by Invariant 4. Missing or malformed state provisionally rejects the ring in the current image. A structurally complete ring additionally validates every candidate and its non-preflushed payload suffix. Failure is also

provisional. By Invariant 6, B is not selectable by ordinary recovery. By Lemma 2 and the group-wide form of Invariant 9, either the preceding canonical Z_0 , independent root, or complete predecessor-ring proof remains recoverable from the alternate slots. Before that predecessor is returned for mutation, Invariant 7 requires an exact independent abort root unless the complete candidate slot has reverted byte-for-byte to its canonical lower-generation pre-attempt source. That source may contain its own exact lower-generation witness; the exceptional image retains no distinguishing byte or witness effect from the admitted candidate generation. If an intervening crash during the abort body makes the candidate complete, Lemma 6 applies and recovery selects the whole new vector. Otherwise exact A consumes every distinguishable rejected generation, and only then do Lemmas 4 and 3 permit later offset reuse without reviving the candidate. Thus the incomplete candidate is invisible in every individual image, while the pre- A rejection classification itself need not be monotone. $\square$

Lemma 6 (Complete prepare is sufficient). *If recovery validates a complete prepared ring with no group-bound final certificate and $\text{IssuedWitness}(G)$ holds for that ring, every participant of the admitted G has its candidate state reconstructible in full.*

Proof. Issuance provenance identifies the decoded participant set and witnesses with the admitted G . The ring supplies the exact candidate root and identity for every participant. Definition 5 validates candidate length, parity, integrity geometry, and ordinary recoverability. By Invariant 5, bytes before each suffix start were durable before its prepared record could be issued. Recovery reads the entire remaining suffix and verifies its exact length and CRC32C. Under the no-collision assumption, all bytes named by every candidate are therefore present. Rewriting all retained roots to M (and removals to T) cannot synthesize a partial candidate. $\square$

Lemma 7 (Final certificate implies an established decision). *An exact group-bound M or T certificate whose witness satisfies $\text{IssuedWitness}(G)$ can exist only after decide(G).*

Proof. There are exactly two finalize guards. Normal execution first joins successful durability obligations for every participant. Recovery first validates the complete exact ring, every eligible candidate, and all required suffix bytes. Either guard establishes decide(G) before any M/T byte is issued. By Invariant 2, neither the older same-slot image, the B prepare, nor an A abort contains $\chi(g_i, M)$ or $\chi(g_i, T)$. An exact M/T header therefore implies that the corresponding guarded final write was issued; it cannot be an exact bitwise reconstruction assembled from pre-decision writes. A crash may tear that later write but cannot reorder its issue event before the guard. The exact adjacent witness binds the certificate to G only under the explicit issuance-provenance premise; an unbound R is excluded by Definition 7. $\square$

Lemma 8 (Installation convergence). *Once a group-bound final certificate exists, recovery can make every still-present exact participant independently final without consulting payload history.*

Proof. Lemma 7 establishes the decision predicate, whose two constructors both prove the candidate payload, so crash-time suffix rehashing is no longer required as a commit test. Each intact witness names the exact legal root spellings. The bitwise transition predicate accepts any subset tear produced by final overwrite and rejects unrelated bytes. Recovery rewrites the deterministic M or T header durably. Within Definition 1’s observation interval, a protocol-generated missing path after finalization was an exact tombstone that has already been unlinked; a missing retained participant is excluded by Limitation 2. After that interval, proof transfer and fresh named-generation identity order any replacement or removal after this group. $\square$

Lemma 9 (Post-finalization namespace independence). *Once UNO may issue the first unlink, any subset of removed-participant names may subsequently be absent without changing the group’s logical new vector. Cleanup never unlinks a retained participant or a recreated incarnation.*

Proof. By Invariants 10 and 11, the first unlink is downstream of an exact durable final root at every participant. A retained participant is never an unlink target and its M root selects the new value independently. For a removed participant, either its exact T remains and Invariant 12 makes it logically absent, or its name is missing and the same absence is represented physically. Thus arbitrary holes among removed names cannot create a mixed vector. Finally, Invariant 13 requires incarnation, witness, and inode equality immediately before unlink, so cleanup for the old decision cannot remove a same-name replacement. Descending traversal only reduces how much garbage later opens or scans must reclaim. $\square$

Lemma 10 (Composition with later namespace operations). *After independent finalization of group G , a later serialized operation may remove or replace any participant of G and thereby create an arbitrary hole in G ’s ring without making recovery observe a state ordered before G .*

Proof. Invariant 11 first transfers G ’s authority from collective ring closure to one independent final record per participant. A later operation is admitted only after that transfer or the exact-ring successor transfer allowed by Invariant 9. If it removes a participant, that effect is ordered after G ; if it recreates the key, the replacement has a fresh named generation (and a fresh V2 incarnation when atomic). Invariant 13 prevents stale cleanup for G from touching that replacement. Every untouched participant still has its independent M or T state. Therefore a broken ring changes only which eager cleanup walk is available, and recovery admits the serial history containing G followed by the later operation. $\square$

Conjecture 1 (UNO group crash atomicity, conditional). *If the transition system is strengthened so every authority-bearing prepared or final witness satisfies IssuedWitness(G), then a conforming implementation satisfies Definition 1: during G ’s protected logical-key interval, every protocol-reachable crash image exposes either G ’s complete old vector or its complete new vector, never a mixed vector. The current one-phase prepare does not satisfy the premise.*

Proof. Conditional on the missing issuance premise, Lemma 1 supplies the remaining protocol invariants. Partition the trace at the crash into three disjoint cases.

No pre-crash decision, and recovery returns old without establishing one: there is no group-bound final certificate. If every candidate slot equals its canonical lower-generation pre-attempt source, including any witness belonging to that source, the prepare left no distinguishable candidate-generation admission effect and the predecessor vector is already authoritative. Otherwise, every image before exact A either lacks complete candidate evidence or is reclassified into the next case. Lemma 5 keeps an incomplete candidate invisible, and no caller mutation, destructive payload change, or slot reuse is enabled during this provisional interval. Proof-preserving predecessor finalization may still run. Exact A then selects the complete predecessor vector, changes only internal generation, and makes that old outcome permanent by Lemma 3. If a torn abort write destroys the recognizable witness without producing complete candidate evidence, Lemma 4 routes the raw non-quiescent slot back through normalization rather than mutable fallback.

No pre-crash decision, and recovery establishes one before exact A : recovery has found a complete ring and validated every candidate and suffix. This includes a ring made valid by the arbitrary subset of an interrupted abort-body write. Lemma 6 proves the complete candidate vector, after which finalization yields the new vector.

A pre-crash decision exists: the normal prepare-barrier join or an earlier recovery already established decide(G). If no exact group-bound certificate exists, no unlink is enabled; all paths and exact witnesses remain. The complete ring still validates roots in any allowed torn transition, and its candidate payload was proved by the decision, so recovery finalizes the group. This covers a crash during the first final write that leaves no exact M/T spelling. If a certificate exists, Lemmas 7 and 8 repair every present participant. If unlink has removed some or all deleted participants, Lemma 9 maps every missing or surviving tombstoned removal to absence while retained M roots select all retained candidates. This includes an all-removal group with no surviving path.

No unlink is enabled during provisional rejection or on the exact- A old path. On either new path, the final-before-unlink guard and Lemma 9 map every removal to logical absence without mixing retained participants. Thus these cases exhaust successful recovery traces and yield only the two vectors in Definition 1. $\square$

Remark 1 (Refinement status). Conjecture 1 is not a theorem of the current one-phase target state machine, and it is not a machine-checked proof of the Rust implementation. Limitation 1 is a target-protocol blocker independent of implementation correspondence. The R13 baseline at 26a532dd6 also has counterexamples to newer- T authority, group-bound certificate selection, rejected-witness reuse, filesystem namespace ownership, and an ineligible integrity candidate. The reviewed branch repairs the namespace-ownership item without changing the storage format; the other counterexamples remain. Current code also relies on, but does not mechanically enforce, the direct-regular-file and no-alias namespace preconditions. Executable correspondence and each repair obligation are recorded in §13.

Corollary 1 (Acknowledged durability). *In a conforming implementation, if `start_apply` returns `Ok`, every later recovery returns the new group vector or a state obtained by serializing later protocol-reachable mutations after that vector.*

Proof. Acknowledgement occurs only after all prepare durability operations succeed. Thus the group has a durable byte-for-byte copy of the actually issued complete ring and `decide( $G$ )` even if no root has yet been materialized; the pre-acknowledgement splice in Limitation 1 is therefore unavailable for G . Finalization preserves that witness, and later groups are serialized on overlapping participants and preserve or transfer the earlier proof before slot reuse. Lemma 10 covers later namespace changes that physically break the earlier ring. This is an acknowledged-prefix claim, not a restoration of the unacknowledged admitted-window conjecture. $\square$

Limitation 3 (Indeterminate admitted window is not atomic). If physical admission occurred but `start_apply` did not return `Ok`, the current one-phase protocol can return a partially new vector. The old-two/new-three trace in Limitation 1 is a concrete counterexample. A validation failure or cancellation proved to precede admission still leaves only old selected states. Code-last prepare, a splice-binding commitment, or a narrowed API guarantee is required before the old-or-new admitted-window claim can be reinstated.

Proposition 1 (No cross-group temporal order). *For disjoint groups G and H , UNO records no durable order between their decisions. If neither has been acknowledged, a crash may recover either group, both, neither, or—until Limitation 1 is repaired—a synthetic subset. An acknowledged group still satisfies Corollary 1 independently.*

Proof. Neither ring contains a link, sequence number, or authority belonging to the other. Per-file barriers impose no cross-file order. Current backends may serialize calls under a process-local namespace lock, but that scheduling fact is not persisted as protocol state. $\square$

9 Progress and resource bounds

9.1 Recovery work

Core independent-root recovery reads two fixed root headers and does not scan payload. A public filesystem open first performs bounded namespace recovery, which may inspect the immutable header and both full 2 KiB witness slots before selecting those roots. An unresolved group followed from one anchor visits exactly n declared links and reads at most 64 MiB of speculative payload across the group. Thus, for total successor-name bytes K , one candidate recovery attempt costs

$$T_{\text{recover}} = O(n + K + 64 \text{ MiB}), \quad M_{\text{recover}} = O(n + K),$$

independent of historical blob length and the number of prior epochs. The 64 MiB term is a byte ceiling, not an allocation requirement; validation streams in bounded chunks. Because one anchor has two slots, an exact-name open may make at most two such candidate attempts, so its speculative-payload ceiling is 128 MiB in the worst case, still a history-independent constant. An abort-authority repair may also materialize the immediately preceding linked group, but that proof is already one of those two slot-local candidates.

A partition scan is not one candidate attempt. Without per-scan group-ID deduplication it may rediscover the same n -member ring from n names, for $O(n^2)$ metadata work and repeated idempotent final-root writes in the worst case. It remains independent of historical payload size, but the tighter one-anchor bound must not be attributed to a full scan.

There is no UNO-specific participant policy cap. The on-disk count is a u32; installation uses chunks of at most 32 concurrent workers without limiting membership. Each edge independently limits its successor partition plus name to 1584 bytes.

9.2 Liveness conditions

Proposition 2 (Conditional completion). *For each decided group G unresolved after restart, suppose that after some time crashes and cancellations cease, required non-removed paths remain available, a surviving member of G is eventually inspected, recovery work and lock acquisition are scheduled fairly, and every issued storage operation eventually succeeds. Then recovery materializes or supersedes G . In a live process, a removal-free decided group may instead remain indefinitely as an active durable carried ring; that is a completed logical decision, not promised physical materialization. After independent finalization, logical completion no longer requires an intact ring; physical tombstone reclamation additionally requires that each lingering name or its partition eventually be inspected.*

This is not availability under media loss. A missing retained participant, a corrupt immutable header, a checksum collision, or a permanently failing barrier may pre-

vent completion. The system fails closed rather than inventing payload. Physical deletion has weaker liveness: a stranded tombstone whose name is never opened and whose partition is never scanned may consume space indefinitely, although it remains logically absent.

9.3 Write and memory cost

Payload bytes are issued when appended and are not copied into a transaction log. The abstract steady-state decision adds one canonical 2 KiB root/link-slot write per dirty participant and one dependency layer of per-file durability operations. The group keeps $O(n + K)$ descriptors in memory, not a payload-sized staging buffer and not an extent map. Large append epochs may cause earlier file barriers, but those barriers only advance immutable payload durability; the payload itself is still written once. Deletion and participant-set changes add the durable M/T finalization writes and, for deletion, unlink plus directory-barrier completion work after the decision. Rejecting a surviving candidate or normalizing a non-quiescent orphan slot during recovery adds one 111-byte abort-body barrier and one one-byte abort-guard barrier before the predecessor becomes writable. If that predecessor was a carried ring, it is first materialized group-wide, adding an earlier barrier layer. Both are rollback-path work, not steady-state commit barriers.

The design does not guarantee that n file barriers become one device flush. It gives the kernel all independent durability obligations without inserting a serial coordinator barrier. Device, filesystem, scheduler, and backend behavior determine actual coalescing. For nonempty work, let m be the number of locked canonical operations in the current Tokio backend. It schedules at most $\min(m, 32)$ chunks, each with at most $\lceil m / \min(m, 32) \rceil$ locked operations, and awaits root writes and retained-file syncs sequentially within a chunk. If chunk j contains d_j actually prepared participants, its staging phase executes at most $2d_j$ such I/O awaits (a deletion’s durable root write combines its obligation). Clean locked operations need not become ring participants, so this bound is intentionally stated in m and d_j , not only in ring size n . The `io_uring` backend uses broader future fanout. “One layer” is therefore the protocol dependency depth, not a measured scheduling claim for every backend.

10 Overlapping groups and two slots

Groups that share a participant serialize because its next generation and alternate slot depend on its current state. Two slots intentionally bound protocol and recovery complexity.

An immediately following group may supersede a carried group without first writing M roots only if:

1. participant keys and incarnations are identical and in the same canonical order;

2. every new base generation is exactly the previous candidate generation; and
3. every new candidate uses the opposite slot.

If the successor group is incomplete after a crash, the preceding complete ring remains in the other slots and is recovered. If the successor ring is complete, recovering it yields a state after the preceding group in the same per-participant generation order. Any different overlapping set is forced to materialize the carried group first. Disjoint sets have no semantic order even though the current filesystem implementation conservatively serializes namespace batches.

11 API semantics

The public distinction between decision and cleanup mirrors the proof boundary:

`start_apply(...)` returns Ok.

The linked decision is durable. A crash must recover the complete new vector. The returned handle may be dropped without canceling recovery.

Awaiting the returned handle succeeds.

In-memory state is active; eager materialization, truncation, and namespace deletion required by that batch have completed.

Validation error before admission.

No candidate/link byte was issued and selected states are unchanged. Invisible payload writes or preflush may already have occurred.

Error or cancellation after admission.

Reopen; do not reuse a poisoned generation. Until Limitation 1 is repaired, a torn one-phase prepare may recover old, new, or a partially new synthetic sub-ring. Cancellation proved to precede admission is old-only.

Committed-only readers observe the selected root. Pending appends are immediately visible through the current open atomic handle but are not crash-durable until direct sync or batch publication. Existing handles to a removed inode remain readable because tombstoning and unlink do not truncate the inode; they cannot authorize mutation of a same-name replacement.

12 Implementation correspondence

Table 3 identifies the executable definitions to which this specification is intended to refine. Function names are used instead of line numbers so ordinary edits do not stale the map.

Specification element	Source module	Executable anchor
roots, slots, transitions	<code>storage/atomic.rs</code>	<code>encode_root</code> , <code>recover_from_root_slots</code> , <code>candidate_root_is_transition</code> , <code>candidate_root_is_batch_transition</code> , <code>candidate_root_is_delete_transition</code>
payload preflush and suffix proof	<code>storage/atomic.rs</code>	<code>PayloadChecksumTracker</code> , <code>validate_payload_checksum</code>
ordinary-blob wrapper	<code>storage/uno.rs</code>	<code>Core</code> , <code>AtomicBlob</code> , <code>stage_batch_commit</code> , <code>activate_batch_commit</code>
canonical operation set	<code>storage/batch.rs</code>	<code>canonicalize_operations</code>
embedded preflight and ring format	<code>storage/batch/coordinator.rs</code>	<code>preflight</code> , <code>encode_link</code> , <code>decode_link</code> , <code>prepare_embedded</code> , <code>prepare_single_publish</code>
ring discovery	same	<code>recover_linked_decision</code> , <code>matching_link</code>
decision recovery	same	<code>recover_embedded</code> , <code>validate_speculative_participant</code>
materialization and eager deletion	same	<code>materialize_decision_participants</code> , <code>unlink_deleted_participants</code> , <code>recover_ordered_delete_frontier</code>
independent tombstone recovery	same	<code>recover_named_embedded</code> , <code>recover_partition_embedded</code> , <code>unlink_exact_tombstone</code>
Tokio batch scheduling	<code>storage/tokio/mod.rs</code> , <code>storage/uno.rs</code>	<code>start_apply_guarded</code> , <code>stage_batch_chunk</code> , <code>Core::sync_batch_commit</code>
io_uring batch scheduling	<code>storage/iouring.rs</code>	<code>start_apply_guarded</code>
filesystem open and scan integration	<code>storage/tokio/mod.rs</code> , <code>storage/iouring.rs</code>	<code>open_versioned_inner</code> , <code>scan</code>
single-participant filesystem paths	<code>storage/tokio/unix.rs</code> , <code>storage/iouring.rs</code>	<code>V2Publisher::publish</code>
public contract	<code>atomic_api.rs</code>	<code>AtomicStorage</code> , <code>AtomicBlob</code> , <code>BatchStorage</code>

Table 3: Specification-to-code correspondence.

13 Implementation refinement status

13.1 Confirmed partial correspondence

At revision 26a532dd6, `finish_embedded_decision` normally makes every present participant independent before eager unlink begins. Exact-name and partition recovery also recognize a current candidate tombstone as absent when that tombstone is the decision they reach, and exact incarnation/inode checks protect a replacement while namespace ownership is held. Focused regressions cover direct non-anchor tombstone open, ordinary reopen, and scan after one peer is recreated. These are useful local properties, but the counterexamples below show that they do not yet compose into full refinement.

13.2 Unchecked namespace-shape preconditions

Assumption 3 requires distinct logical keys to name distinct inodes. Without it, two participants can be hard links to one file: both local candidate writes target the same slot, both durability calls can succeed, and the last write leaves only one witness. Acknowledgement would then be followed by recovery of the old alternate root because no two-member ring remains.

The runtime creates distinct inodes for distinct names and its exclusive-directory contract rules out external namespace mutation during operation, but revision 26a532dd6 does not detect a hard-link alias already present at open or compare participant device/inode identities during batch preflight. The conditional conjecture therefore applies only when the owned namespace was

created through conforming operations and has not been externally aliased. If arbitrary pre-populated directories must be accepted, initialization or admission must reject aliases and a two-hard-link acknowledge/crash regression is required.

There is a second mismatch for symbolic links. Ring discovery and exact namespace recovery use `symlink_metadata` and `O_NOFOLLOW`, so a symlink is not a participant. The ordinary existing-file open paths in both Tokio and io_uring instead use `OpenOptions::open` without `O_NOFOLLOW`; on Unix they follow a pre-existing symlink. Thus a symlink to a valid atomic backing inode can be opened and updated on the live path but skipped by the recovery path.

A one-participant execution suffices: open the symlink as an atomic blob, acknowledge a self-ring group whose prepared candidate is otherwise valid, crash before materialization, then reopen the same name. Namespace recovery declines the symlink, while ordinary open follows it and can select the old alternate root. The acknowledged candidate is lost. A conforming backend must inspect the directory entry and open it with no-follow semantics as one serialized admission step, then bind the opened descriptor to the inspected device/inode identity. A symlink-to-valid-V2 acknowledge/crash/reopen regression is required on both filesystem backends.

13.3 Target blocker: exact witness need not have been issued

Limitation 1 is not an R13/R14 correspondence mismatch: it invalidates the one-phase proposed target itself. A full-slot prepare may tear into an exact current-generation B root plus a checksummed witness whose topology came

from the previous same-parity group and whose candidate fields came from the new group. Recovery has no separate manifest with which to distinguish that exact synthetic ring. The two-participant composed model starts from zero witness bytes and therefore underapproximates this trace.

The independently reconstructed old-two/new-three fixture satisfies consecutive lineage, distinct group identifiers and incarnations, canonical slot and link parsing, payload validation, and the paper’s no-CRC-collision event. It yields a mixed observation of the actually admitted three-member group. Consequently Conjecture 1, the indeterminate admitted-window guarantee, and any positive model result that assumes witness intent are blocked.

The correctness-first repair is a code-last prepare: make all non-guard slot bytes and candidate payload/length state durable, then issue the one-byte B guard and make it durable. This gives the guard disk-observable provenance for the exact witness but requires two ordered per-file durability layers. Preserving one layer instead requires a new splice-binding cryptographic commitment and a stronger computational assumption. Selecting between those contracts is an unresolved architecture decision; no Rust implementation change for either repair is claimed here.

13.4 Blocker: newer tombstone can lose to an older ring

Stale-witness suppression in `candidate_has_embedded_batch_witness` recognizes newer exact R/M roots but not a newer T . Consequently `recover_embedded` can choose an older complete retained ring before `recover_materialized_witnesses` gives the tombstone authority.

A reachable acknowledged counterexample is:

1. materialize an all-retained group $H = \{A, C\}$;
2. commit deletion group $D = \{A, B\}$, producing newer T_A, T_B ;
3. unlink B and crash before unlinking A ; and
4. open A . The D ring is broken by missing B , while H remains complete. Because T_A does not suppress H ’s older witness, recovery may reinstall old M_A and resurrect A .

This violates the target recovery-selection rule and acknowledged deletion safety. A repair must make the newest exact T suppress every older witness and must resolve that tombstone before an older non-delete verdict. The sequence above needs atomic-open, ordinary-open, and scan regressions.

13.5 Blocker: R13 permits exact unisued authority synthesis

The checked-in R13 spellings use one generation-independent magic per state. Under arbitrary byte-subset persistence, that is not enough to prove that a checksum-valid $M/T/R$ header was ever issued. A candidate slot

can already contain an older same-parity M header. A torn B prepare can retain the old M magic while taking the new generation and logical fields. A later torn rollback write can supply the remaining root-checksum bytes while leaving the exact new witness in place. For concrete caller-tag values, the resulting 112 bytes equal the canonical new M header even though no decision or M write occurred; this is exact reconstruction, not a CRC collision. Certificate-first recovery would then publish one participant while its peer rolls back.

The R14 target closes that provenance gap with Invariant 2. A single spelling byte jointly encodes the state and generation modulo three. The previous use of the same slot was two generations earlier and therefore has another color; B , A , R , M , and T within the new color have distinct byte values. Consequently a crash may preserve any subset of all other root bytes and checksum bytes, but an exact state whose guard write was never issued is unreachable. This format change is required before Lemma 7 applies to the implementation. Regressions must enumerate every subset of the abstract spelling/body/checksum/witness cells across a torn prepare, repeated abort recovery, finalization, and subsequent slot reuse.

13.6 Blocker: ordinary roots are not group certificates

The current recovery path also treats `candidate_is_committed` (R) as evidence that group installation started, even though R contains no group identifier. A rejected stale ring can remain beside a later direct publication at the same generation. If the later publication happens to reproduce the linked candidate’s root fields, exact R_i can masquerade as a certificate for the stale group and bypass a peer’s failed suffix validation.

The target protocol therefore uses only the generation-colored, group-bound M/T pair in Definition 7. Rejection consumes the stale generation through an A root before a persistent direct $P \rightarrow R$ publisher may proceed, and recovery never treats its R as group evidence. The checked-in generic direct publisher and the current $R/M/T$ installation test do not jointly prove those properties. A regression must reject a batch for a bad peer suffix, attempt direct republication, crash at each abort/publication boundary, and verify that the stale batch is still rejected.

13.7 Blocker: rejected witnesses survive payload reuse

When `recover_embedded` finds a structurally complete ring whose speculative suffix fails validation, it currently continues to an older authority without durably invalidating any wrapper. Ordinary recovery may then truncate to the predecessor length while leaving that root-page witness intact. The in-memory write stamps used during the original preparation are not persistent evidence.

A two-crash counterexample uses one participant. Group G appends $P \parallel S$, durably preflushes P , and describes only S as its speculative suffix. A crash leaves the exact prepared witness but not a valid S , so recovery rejects G and exposes the predecessor. A later append reuses the same offsets for $P' \parallel S$, where $P' \neq P$, and crashes before admitting its own root. The old witness can now validate S and treats the prefix as preflushed, causing recovery to publish $P' \parallel S$ as G 's candidate even though G proposed $P \parallel S$.

The target reject-and-consume transition prevents this by first making any linked predecessor independent, then consuming the rejected generation with a generation-colored, code-last A root. Writing the complete A header in one unordered phase would still be insufficient: its surviving A guard could combine with candidate body bytes and mixed checksum bytes to form a valid A -spelled candidate value. The non-guard barrier followed by the one-byte guard barrier removes that source combination. Merely zeroing a wrapper is not sufficient under repeated arbitrary-subset slot rewrites. The current implementation needs that abort protocol and regressions covering invalid-suffix rollback, changed-prefix reuse, same-slot reuse after an intervening generation, and repeated crashes.

The abort root alone is insufficient unless recovery can recognize a crash *during* its write. The current batch witness write is intentionally short rather than a canonical zero-padded full-slot prepare, and ordinary fallback has no raw-slot orphan classification. A torn recovery write can therefore destroy the recognizable witness and remain invalid. The target additionally requires canonical full-slot prepare images, Definition 9's wrapper-independent normalization branch, the colored-guard issuance invariant, and code-last abort ordering. A regression must tear every non-guard cell in the first phase and the guard in the second, crash again during retry, then reuse the payload and slot; no trace may revive the rejected group or expose a value that no admitted operation built.

13.8 Reviewed cancellation repair: self-driving namespace ownership

The R13 baseline originally kept the Tokio namespace guard in the cancelable observer for `open`, `scan`, and `remove`. Already admitted recovery, unlink, directory-barrier, or blocking `set_len` work could therefore outlive that guard. The synchronous atomic-write path also awaited carried-decision materialization inline in both Tokio and `io_uring`. Those gaps admitted three concrete stale-work races: pathname cleanup against a recreated name, reuse of the removed generation between unlink and invalidation, and an older materializer or recovery truncate overwriting a later acknowledged publication.

The reviewed branch makes these operations self-driving after namespace admission. Tokio `open_versioned_inner`, `scan`, and `remove` acquire an owned namespace guard and move it, the storage

handle, and owned arguments into a spawned backend task. The caller awaits only that task's result. Dropping the observer no longer stops the worker or releases authority: `open` retains it through named recovery, any physical-surplus truncate, and shared V2-state installation; `scan` retains it through partition recovery and creation-file cleanup; and `remove` retains it through requested unlink, the appropriate directory barrier, generation/V2-state invalidation, and sidecar cleanup.

The runtime-agnostic UNO wrapper now routes an admitted synchronous publication through the same self-driving `Publisher::spawn` mechanism already used by `start_sync`. The spawned task owns the armed mutation state. Consequently a canceled `write_at_sync` observer cannot release the publisher's namespace authority while an older carried group is being materialized. This shared repair applies to the Tokio and `io_uring` publishers; a spawn failure drops the armed guard and poisons that generation.

Deterministic regressions pause and cancel Tokio `open`, `scan`, and `remove` with blocking work queued; pause exact removal after unlink but before directory durability and invalidation; pause the exact recovery `set_len`, then reopen and acknowledge an append; and pause carried-group materialization during a direct synchronous publication. The last regression also runs against the Linux `io_uring` backend. Each test proves that namespace authority remains unavailable while the parked worker is outstanding and that a later reopen derives the expected bytes or a fresh generation. These tests close the cancellation-ownership races above for the reviewed code.

This repair does not weaken Assumption 5. Runtime shutdown, task panic, or an indeterminate mutable I/O error still requires the affected instance to stop and recover from disk; self-driving work is not a promise that an observer canceled after admission can safely ignore that assumption. Nor does this R13 implementation acquire the proposed R14 generation-colored guard, code-last abort normalization, or the other format and integrity repairs in this section. It is therefore not a refinement proof from the checked-in implementation to the proposed R14 conditions or conditional conjecture.

13.9 Availability deviation: missing successor partition

After group-wide independent finalization, a later serialized whole-partition removal may delete a successor's parent directory. The target recovery rule then stops stale frontier traversal and uses the local M/T authority. The current `recover_ordered_delete_frontier` instead attempts `sync_directory` on the missing successor partition and propagates `ENOENT`. This does not return an incorrect vector, but it can make an independently recoverable surviving blob fail to open until the partition is recreated. A cross-partition finalization/removal regression and a fail-closed-but-available handling rule are required.

13.10 Known integrity-boundary issue

There is also a separate edge case in fixed-size integrity binding. Starting unbound, append exactly N bytes with `Continue`, then call an empty `Chunked(N)` append. Preparation recognizes that the existing unit should close, but the empty data loop never invokes the sealing step. A chunked root can then name an unfinished unit of length exactly N , which reopen rejects because a chunked tail must be shorter than N . This violates Definition 5: if such a root is acknowledged in a group, the implementation cannot establish the candidate value on reopen. It is therefore a refinement blocker for both per-blob recovery and any batch containing that candidate, not merely an orthogonal integrity concern. It needs a regression before repair.

Rewind has a second reachable geometry gap. A caller can supply a non-current completed `IntegrityUnit` with zero length or an offset/width that does not satisfy the public scheme. The current source-validation path may accept four zero bytes as the footer for an empty unit because $\text{CRC32C}(\epsilon) = 0$, then stage a chunked root whose unfinished start is misaligned. Batch prepare does not independently reject that root, so it can be acknowledged even though ordinary reopen rejects its geometry and falls back. Every supplied non-current unit must pass the same nonempty and scheme-geometry validation as a completed unit before payload I/O or mutation, and batch admission must reject any root ordinary recovery would reject. Zero-length, wrong-offset, and wrong-width rewind regressions are required for direct sync and group publication.

14 Evidence and validation obligations

The current tests exercise the principal proof partitions:

- arbitrary retained subsets of payload writes and arbitrary non-prefix prepared-root bytes cannot publish an incomplete group;
- complete rings larger than the 32-worker fanout recover, while one missing link at either side of a worker boundary rolls back;
- a missing intended deletion before finalization is not treated as a commit decision;
- every mixed-removal mask and every descending durable-unlink frontier is completed from ordinal zero;
- a surviving non-anchor tombstone cannot resurrect old payload through atomic or ordinary exact-name open;
- partition recovery does not enumerate a stranded tombstone after another participant is recreated;
- arbitrary bitwise mixtures of prepared, materialized, and tombstone roots are repaired once an exact group-bound M/T witness proves the decision;
- stale replay preserves a recreated same-name incarnation;
- normal recovery reads bounded metadata rather than

historical payload; and

- large pending epochs preflush so unresolved group validation remains within 64 MiB.

These tests are finite evidence, not a proof of the Rust implementation.

14.1 Executable Quint/Apalache model

The companion `docs/uno-protocol.qnt` uses Quint 0.32.0 and Apalache 0.56.1; the pinned image and exact command ledger are in `docs/uno-model-checking.md`. The file contains three complementary modules. The fast `uno_slot` abstraction separates generation, the one-byte state/color guard, logical body, all four CRC bytes, and witness into decoder-relevant addressed cells. The larger `uno_group` control abstraction chooses one to three participants and independently assigns append, rewind, or remove, including payload validation, interruptible abort/-final repair, post-decision rewind, and partial unlink.

The `uno_composed` bridge fixes exactly two participants and derives recovery observations from on-disk evidence. It uses concrete offset-indexed 112-byte representative R14 headers, all four CRC bytes, generation-1 same-slot source/generation-2 ordinary- R predecessor/generation-3 candidate lineage, a seven-byte payload, and a compact exact 16-byte reciprocal-ring witness abstraction. It defines pair-fixed cuts for all nine ordered append/rewind/remove combinations. Prepare, ordinary- R , abort-body, abort-guard, final, witness, payload, and unlink outcomes retain independent survivor masks; physical file-length persistence is an independent old/new scalar abstraction. The full crash actions admit an arbitrary subset of every addressed root or repair write. Some focused step relations instead use predicate-specific dependency projections or staged successful-barrier summaries; a result applies only to its named step relation. The 111-byte abort body and one-byte guard are separate barrier phases, and either issued unlink may independently survive. Recovery re-decodes the resulting roots, witnesses, payload, and namespace rather than consulting an intended outcome. In particular, it represents provisional pre- A rejection, reconsideration after a torn abort body, exact- A precedence, interrupted final repair, and observations stored from the disk decoder.

The composed bridge deliberately initializes each generation-three witness overlay from an all-zero image and fixes the intended ring to two participants. It therefore cannot express the confirmed generation-one $H = \{a, b\}$ to generation-three $G = \{a, b, c\}$ witness splice in Limitation 1. Positive results from this module do not establish `IssuedWitness(G)` and cannot validate Conjecture 1. A repair model must include the exact older wrapped witnesses, a differing later participant set, independent survival of every wrapper/link/checksum byte, and disk observations for every originally admitted participant.

Validation is paused at a non-final checkpoint. The current model artifact (SHA-256 `dde48576...2b75965`; full

digest in the companion note) has not been typechecked since a fresh full-diff audit corrected its generation-2 predecessor fixture from the M guard and CRCs to the claimed ordinary- R encoding. Its immediate predecessor typechecked. Apache found the intended deliberately generation-broken counterexample and completed several pair-fixed staged obligations on earlier development revisions, but the file changed afterward. The exact- A run was interrupted when execution was paused, and the current-hash all-pair matrix has not been run. Accordingly, those results are development evidence, not final validation; the exact bounds, elapsed times, OOM/interrupted runs, and pending commands are recorded candidly in the companion note.

This remains bounded model checking, not an arbitrary- n proof, an ext4/JBD2 model, or a Rust refinement. The compact fixed witness does not validate the complete physical L14 codec or padding; the composed model covers one two-participant, three-generation window and a tiny representative payload. Its current old-observation cut requires both participants to be locally quiescent or exact A ; it has not checked the intermediate paper route where one exact A suppresses the ring and a peer's stale B slot is normalized only before peer mutability. It does not model duplicate-inode or symlink admission, rename/recreation and stale inode identity, journal replay, delayed allocation, concrete syscall order, or cancellation ownership. It also does not establish public API reachability, multi-generation source-window wraparound, the newer-tombstone/older-ring implementation trace, stale-ring direct- R aliasing, rejected-witness payload reuse across multiple later generations, or the fixed-chunk and rewind integrity blockers. Those require codec tests, filesystem refinement tests, and continuing Rust correspondence work beyond the reviewed cancellation-ownership repair.

14.2 Audit checklist

The state-machine proof fails if any of the following becomes reachable:

1. a checksummed exact witness can be reconstructed from older and issued-new slot bytes without being the exact witness issued by the candidate-generation prepare;
2. an M/T final write can be issued before either the all-prepare barrier join or complete recovery validation establishes `decide( $G$ )`;
3. a root format permits an exact $R/M/T$ generation- g spelling to be assembled without issuing that generation-colored state byte;
4. $\chi(g, A)$ can be issued before every other byte of $A_g(O)$ is barrier-established, allowing an abort-spelled candidate body to validate;
5. recovery treats a failed validation as durable rejection, returns the predecessor for caller mutation or permits destructive payload/slot reuse while a distinguishable rejected generation lacks exact A , or resumes abort

- publication after a crash without first reconsidering complete candidate and payload evidence;
6. an ordinary R can be accepted as a group certificate without a group-bound spelling;
7. final installation can overwrite witness bytes or another slot;
8. a non-eligible candidate can be admitted or acknowledged;
9. ordinary root recovery begins accepting B directly;
10. an unlink can occur before every participant has an independent durable final root;
11. an exact current-incarnation tombstone can fall through to an older ordinary root or appear in a live namespace scan;
12. cleanup can unlink a retained participant or a same-name replacement without exact incarnation, witness, tombstone, and inode validation;
13. admitted cleanup can outlive the namespace guard that excludes recreation, or can unlink by pathname after its exact-identity validation is no longer serialized;
14. canceled inline publication can leave an admitted materialization job able to overwrite a slot after a newer authority has reused it;
15. canceled ordinary recovery can leave a detached truncate able to remove payload after a later publication on the same inode;
16. root reuse removes any needed witness before group-wide independent finalization or a complete exact-ring successor decision;
17. a stale witness can match a different incarnation or survive authority from a later visible $R/M/T$ generation;
18. a rejected generation can remain selectable when dependent payload offsets or its slot are reused, rather than being consumed by a newer independent abort authority;
19. recovery can return a mutable predecessor while its opposite raw slot is non-quiescent, including after a torn abort whose wrapper no longer validates;
20. a rewind truncate can land before the batch decision while the old vector remains a legal recovery outcome;
21. preflush credit survives reuse of offsets after rewind;
22. a participant becomes batch-eligible before its name, header, and incarnation are durable;
23. a symbolic-link path can be admitted as a participant or followed differently by live open and namespace recovery;
24. two distinct live logical keys can alias one device/inode identity;
25. a backend reports durability before its ordinary `Blob::sync` contract is met; or
26. external namespace mutation is admitted without a stronger missing-file model.

15 Non-goals and known boundaries

- **No current admitted-window atomicity claim.** The one-phase target remains subject to Limitation 1; only the acknowledged-prefix result is retained while the prepare-provenance architecture choice is open.
- **No global WAL or total order.** This avoids a global durable lock and volume-like layer, but callers must represent temporal dependencies explicitly.
- **No random overwrite atomicity.** Supporting it requires copy-on-write mappings, undo data, or another log, together with reclamation and recovery metadata.
- **No atomic create in the current batch API.** Exact current incarnations must already exist before ring construction.
- **No Byzantine disk defense.** CRC32C cannot authenticate local state; an attacker able to alter data can also alter checksums.
- **No symbolic-link or hard-link participant semantics.** A batch-capable key must be a direct regular-file entry, and distinct logical keys must not name one inode. A backend accepting arbitrary pre-populated namespaces must detect and reject either shape.
- **No recovery from independent retained-file loss.** See Limitation 2.
- **No prompt reclamation guarantee after a broken deletion ring.** Exact open and partition scan remove independently recognizable tombstones, but an untouched name may retain its tombstone indefinitely.
- **No claim that concurrent fsync calls collapse physically.** UNO removes serial protocol depth; it does not control a device’s flush scheduler.

16 Conclusion

UNO’s commit authority is a closed set of local facts rather than a separate global record. That choice is intended to give append-oriented structures write-once payload, direct per-file reads, and recovery work independent of historical blob size. The central proof obligation is nonstandard: complete-ring validation must survive arbitrary subset tears, distinct logical participants must not alias one inode, and deletion must transfer authority from the ring to independently meaningful final roots before destroying any link.

The current one-durability-layer target does not establish that obligation. Generation coloring proves issuance of a root state, but a checksummed adjacent witness can be an exact never-issued old/new mosaic. The confirmed old-two/new-three trace therefore invalidates unconditional admitted-window atomicity and leaves Conjecture 1 blocked. Alternating slots, prepared invisibility, payload coverage, abort ordering, final-before-unlink materialization, tombstone dominance, and incarnation-safe cleanup remain necessary, but they are not sufficient with-

out exact-witness issuance provenance.

The refinement audit also found independent implementation gaps. Revision 26a532dd6 still allows a newer tombstone to lose to an older complete ring, accepts an unbound ordinary R as group evidence, uses generation-independent R13 spellings that permit exact unissued-root reconstruction, can reactivate a rejected witness after payload-offset reuse, can let admitted namespace work or an ordinary recovery truncate outlive its guard, and can construct noncanonical fixed-chunk or rewind candidates. The reviewed branch repairs the cancellation-ownership item only. It does not enforce the conditional conjecture’s direct-regular-file and no-alias preconditions for a pre-populated directory, and it does not repair the target witness-provenance blocker. No claim is made that R13 satisfies the proposed R14 conditions.

The unresolved trade is explicit. Code-last prepare retains the existing noncryptographic crash model but adds a second ordered durability layer. A splice-binding commitment may preserve one layer but changes the format and theorem assumptions. Narrowing the guarantee preserves the current one-phase shape but gives up atomicity for admitted, unacknowledged groups. Until one of those paths is selected and modeled, UNO has an acknowledged-prefix result and a conditional per-group target, not a complete crash-atomicity proof.

A Compact state-transition summary

For one retained participant, the common filesystem path is

$$R_g \parallel B_{g+1} \xrightarrow[\text{complete ring}]{\text{recover or carry}} R_g \parallel M_{g+1}.$$

For removal it is

$$R_g \parallel B_{g+1} \longrightarrow R_g \parallel T_{g+1} \longrightarrow \perp.$$

For a provisionally rejected candidate, after independently finalizing any linked predecessor, recovery attempts

$$\text{Authority}_g \parallel B_{g+1} \longrightarrow \text{Authority}_g \parallel A_{g+1}.$$

Here $\pi(A_{g+1}) = \pi(\text{Authority}_g)$. Recovery first makes all non-guard A bytes durable, then publishes the one-byte A guard; the stale witness may remain but is suppressed by A . The arrow is provisional until exact A_{g+1} exists. If either phase crashes, restart re-decodes the complete raw slot, witness, ring, and payload. A newly complete candidate diverts to the decision and new-vector path; a non-quiescent image without complete evidence retries the code-last A_{g+1} transition. Only exact A_{g+1} selects a mutable predecessor and makes the old outcome irreversible. This reconsideration/orphan branch is what makes the arrow sound under repeated crashes. Any arrow may crash after changing an arbitrary subset of addressed bytes. The wrapped local link remains exact after the prepare barrier and supplies the complete source

and destination spellings for repair. Before that barrier, however, a checksummed exact link need not have been issued; Limitation 1 blocks treating such a ring as the admitted group’s decision. A direct non-batch correctness-first publisher may use $P_{g+1} \rightarrow R_{g+1}$ only outside batch-capable persistent media or after proof transfer has resolved decided evidence and abort publication has consumed rejected evidence; R never proves a linked group. The participant-linked filesystem hot path uses B and normally materializes to M only when proof transfer is required.

complete append by selecting the old length because old payload bytes remain untouched. It can roll forward a complete append because candidate bytes are already at final offsets. Atomic random overwrite therefore requires persistent undo bytes, copy-on-write mappings, or an external log, reintroducing duplication and reclamation.

B Guarantee matrix for missing paths

Observed state	Interpretation	Required recovery action
missing successor, prepared anchor only	incomplete ring; absence is not a vote	resolve predecessor, publish abort authority, expose old roots
missing path after independent finalization, before a later target-key mutation	protocol-generated absence can only be a removed participant	use surviving M/T roots independently; cleanup may be deferred
missing successor on exact ordinal-zero chain	optional descending cleanup frontier	synchronize observed absence; reclaim reachable lower tombstones
surviving exact tombstone at any ordinal	independently logical deletion	omit from scan/open; never select fallback root
ordinal zero absent or ring non-prefix	no special safety meaning after independent finalization	retained M roots and surviving T roots stand alone
same key, different V2 incarnation or V1 inode	recreated path, not successor	never unlink replacement
missing expected incarnation after a later mutation	state serialized after the older group	follow the later operation; never treat the hole as a prepare vote
missing retained file due external action	outside protocol model	no atomicity or reconstruction guarantee

C Why a checksum alone is insufficient for overwrite

A checksum can distinguish an expected byte string from most torn strings, but it cannot restore bytes that an in-place overwrite destroyed. UNO can roll back an in-